

基于统计建模与元学习的 NIPT 检测决策优化与异常识别

摘要

随着精准医疗技术的发展，无创产前检测（NIPT）作为染色体异常筛查的关键，其检测精度和时点选择影响着临床效果和孕妇安全。然而，现有检测一方面容易忽略孕妇 BMI 等个体差异；另一方面如果采血时点过早可能因胎儿游离 DNA 浓度过低导致检测失败率升高，过晚则易引发临床安全。

针对问题一，首先通过 Pearson 相关性分析，识别出孕妇个体差异是影响检测效果的关键因素。进而，采用改进的线性混合效应模型（LMM）结合限制性最大似然估计（REML）方法，有效处理重复测量数据的层次结构特征。模型结果表明在可检测孕周范围内，Y 染色体浓度与孕周呈正相关，且增长速度越来越快，与 BMI 和身高呈显著负向关系。模型通过了随机效应必要性、随机截距正态性、同方差性检验。组内相关系数 ICC 证实 74.3% 的总变异源于孕妇间个体差异，相比传统回归模型 AIC 值提升 17.2%。

针对问题二，首先对数据进行聚合得到 267 位孕妇的完整情况。采用改良的高斯过程回归（GPR）与三阶段分层决策流程预测每位孕妇的最早达标孕周，进而构建了一个基于 BMI 分组和最佳 NIPT 时点决策的风险最小化优化模型，结合遗传算法（GA）实现分组边界与检测时点的全局优化。核心结果显示三分组策略为最优解，而 BMI 分界点随准确率要求动态调整。检验存在误差，对误差的容忍度越低，推荐时点会越靠后，50% 准确率下推荐时点为 11.00-12.00 孕周，均为低风险，而 90% 以上准确率要求会将最佳时点推迟到 16-20 周甚至更晚。

针对问题三，模型从依赖单一 BMI 变量变为处理多维异构特征。本文引入生存分析框架，预测孕妇“随时间变化的染色体浓度达标的概率函数”，构建混合深度生存网络。在数据处理上，通过多层感知机（MLP）处理静态特征、长短期记忆网络（LSTM）处理时序特征，最终由 DeepHit 生存分析头输出高度个体化的概率曲线。核心结果表明该模型预测精度达 94.7%。最后再对多维数据建立风险最小化优化模型，结合遗传算法（GA）实现全局优化，得到了不同误差下的分组结果，并通过多因素分析发现影响最佳时点的主要静态指标是年龄和 BMI。

针对问题四的女胎异常判定方法构建，为了解决数据总数少且不平衡，本文采用基于 LightGBM 和 Optuna 的堆叠集成模型框架以充分利用数据。首先，采用 LightGBM+Optuna 构建 13、18、21 号染色体异常的专用判定模型，学习并构建造成各个染色体自身异常的元特征，并进一步学习可能存在的交互作用。最终将元特征与原始数据进行堆叠聚合，训练出一个能够准确判定异常的元模型。结果显示该方法对女胎健康/异常二分类准确率达 99.7%，在 3 组独立验证集上误差均低于 1.5%，并且在准确判断具体异常症状时，各项指标仍超出 70%。

关键词：NIPT 检测优化 线性混合效应模型 高斯过程回归 遗传算法 混合深度生存网络

一、 问题重述

1.1 问题背景

随着精准医疗理念的深入发展和分子生物学技术的不断进步，无创产前检测 (Non-invasive Prenatal Testing, NIPT) 已成为现代产前诊断的重要技术手段。作为一种基于母体血液中胎儿游离 DNA 检测的新兴技术，NIPT 在唐氏综合征等常见染色体异常筛查中展现出显著优势，目前已在全球范围内得到快速推广。

然而当前临床实践中 NIPT 检测面临检测时点选择面临技术可行性与临床安全性的矛盾：过早检测可因胎儿游离 DNA 浓度不足导致失败率升高，过晚检测虽浓度充足但错过早期干预的黄金窗口，且未能充分考虑孕妇的个体差异。研究表明，cffDNA 浓度受孕周、母体 BMI 等因素显著影响。Wang 等人 (2013) 的研究证实 cffDNA 浓度与孕周呈正相关，孕 10-21 周每周升高 0.1%，21 周后增长至每周 1%。同时，Ashoor 等人 (2012) 的研究发现：孕妇体重与 cffDNA 浓度呈负相关，体重增加导致血容量扩张稀释胎儿 DNA 浓度，肥胖孕妇 NIPT 检测失败风险更高。

此外，对于女胎异常的判定，由于缺乏 Y 染色体，现有方法主要依赖 13、18、21 号染色体的 Z 值分析，但在复杂遗传背景和样本分布不均衡情况下的诊断准确性仍有待提升。因此，构建基于多因素分析的个体化 NIPT 检测时点优化模型，以及寻找更加精准的胎儿异常判定方法，已成为当前产前诊断领域亟待解决的关键科学问题。

1.2 问题提出

为使 NIPT 检测在今后的临床应用中获得更高的检测准确性和更好的风险控制效果，现需结合实际情况与附件所给信息建立数学模型，分析以下问题：

- **问题一：**分析胎儿 Y 染色体浓度与孕妇的孕周数和 BMI 等指标的相关特性，给出相应的关系模型，并检验其显著性。
- **问题二：**临床证明，男胎孕妇的 BMI 是影响胎儿 Y 染色体浓度达标时间的主要因素。试对男胎孕妇的 BMI 进行合理分组，给出每组的 BMI 区间和最佳 NIPT 时点，使得孕妇可能的潜在风险最小，并分析检测误差对结果的影响。
- **问题三：**综合考虑身高、体重、年龄等多种因素及检测误差，根据男胎孕妇的 BMI 给出合理分组以及每组的最佳 NIPT 时点，使得在不同 Y 染色体浓度达标比例要求下孕妇潜在风险最小，并分析检测误差对结果的影响。
- **问题四：**针对女胎异常判定问题，试以女胎孕妇的 21 号、18 号和 13 号染色体非整倍体为判定结果，综合考虑 X 染色体、21 号/18 号/13 号染色体

Z 值、GC 含量、读段数及相关比例, BMI 指标等因素, 给出女胎异常判定方法。

二、 问题分析

2.1 问题一的分析

针对问题一, 首先要分析胎儿 Y 染色体浓度与孕妇的孕周数和 BMI 等指标的相关性。接着需要给出相应的关系模型, 并检验其显著性。鉴于数据存在重复测量, 且个体间和个体内变异的层次结构特点, 研究选择线性混合效应模型 (LMM) 作为核心建模工具, 并采用受限最大似然估计 (REML) 进行参数估计。模型结果显示, 回归系数均显著, 且 Wald² 检验表明固定效应具有联合显著性。此外, 研究结合 AIC/BIC 准则进行模型比较, 并进行随机截距正态性检验与同方差性检验, 以全面验证模型假设的合理性。

2.2 问题二的分析

首先, 题干明确指出 BMI 是影响达标时间的主要因素, 因此需要在 BMI 这一连续变量上寻找最优的分割点。之后, 需要为每个独立的 BMI 分层群体计算出一个能使整体潜在风险最小化的最佳 NIPT 检测时点。并且再检误差不同的情况下, 分组边界及最佳时点均可能不同。

为此, 需构建一个两阶段的求解框架。第一阶段考虑到原始观测数据的稀疏性与噪声干扰, 先为每位孕妇推算出其完整的达标时间 (T_{attain})。本文选择高斯过程回归 (GPR) 作为核心预测工具, 推算出孕妇 T_{attain} 的后验分布。第二阶段将问题形式化为一个组合优化问题, 并选择遗传算法 (GA) 进行优化, 得到最优的 BMI 区间及其对应的静态最佳 NIPT 时点。

2.3 问题三的分析

问题三引入了更多维度, 模型必须从依赖单一的 BMI 变量升级为能够综合处理多维异构特征的复杂模型。其次, 决策目标需要构建一个与“达标比例”动态挂钩的、连续的 NIPT 策略函数。

为此引入生存分析作为核心理论框架, 将预测目标转变为“随时间变化的累积达标概率函数”, 契合“达标比例”的动态要求, 并能科学处理观测删失数据。在此框架下, 我们选择构建一个混合深度生存网络作为核心代理模型, 预测达标时点。而后再建立风险模型, 并用遗传算法解出分组与最佳时点。

2.4 问题四的分析

问题四旨在建立数据驱动的女胎异常判定方法。本文构建了基于 LightGBM 的堆叠集成诊断模型, 采用两阶段判断流程。首先进行多专家模型构建与特征融合, 考虑到女胎 NIPT 数据中各指标间存在复杂相关性, 以及单一方法对复合异常识

别能力的局限性，采用四个专项模型协同架构：以交互影响模型及三个专项模型作为核心模块，结合关键原始特征构建混合特征空间。在此基础上，进行元学习优化与逻辑修正，最终得到优化的女胎染色体异常判定模型。

针对以上问题，本文采用了以下整体思路框架进行系统性和分析：

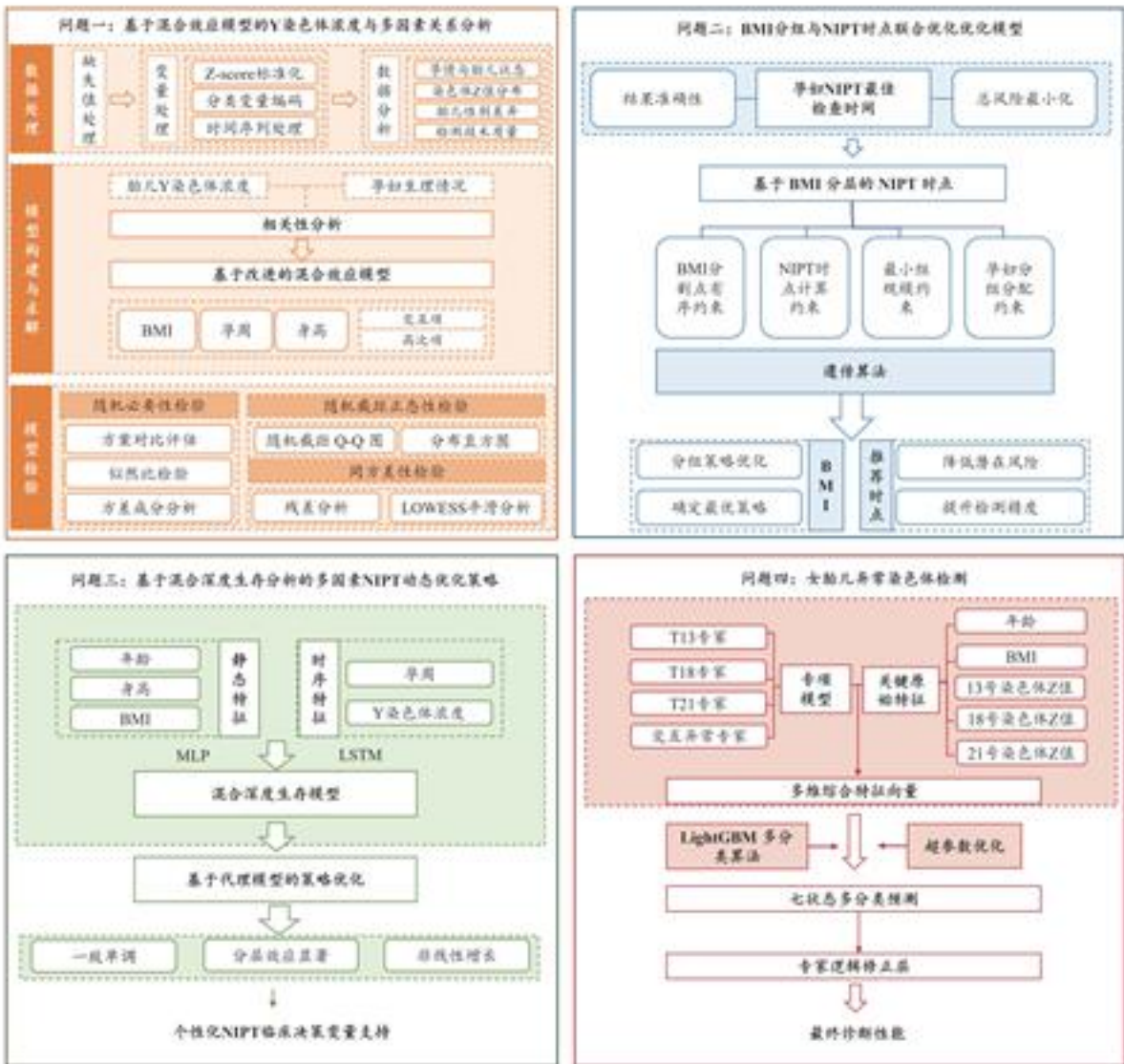


图1 总体思路框架

三、 模型假设

1. 孕妇虽然在年龄、BMI、孕情等方面存在个体差异，但这些差异在统计学上是可建模和可预测的，不存在极端异常个体影响整体模型效果。
2. NIPT 检测的临床有效性阈值普遍被认定为胎儿 Y 染色体浓度达到或超过 4%，若低于该阈值，检测结果的准确性则难以满足要求。
3. 对于同一孕妇的多次检测，假设每次检测在技术上相互独立，检测误差符合正态分布，可通过统计方法进行综合评估。
4. 所有检测数据都源自标准化的实验室流程，具备统一的质量控制标准，测序深度、比对质量等技术参数在可接受范围内变化。

四、 符号说明

符号	说明	单位
Y_i	第 i 个样本的 Y 染色体浓度	%
X_i	第 i 个样本的 X 染色体浓度	%
$Z_{X,i}$	第 i 个样本的 X 染色体 Z 值	-
$Z_{13,i}$	第 i 个样本的 13 号染色体 Z 值	-
$Z_{18,i}$	第 i 个样本的 18 号染色体 Z 值	-
$Z_{21,i}$	第 i 个样本的 21 号染色体 Z 值	-
$Ratio_i$	第 i 个样本的唯一比对读段数比例	-
$Reads_i$	第 i 个样本的唯一比对读段数	个
t_i	第 i 个样本的检测时间	周
BMI_i	第 i 个样本的体质质量指数	kg/m ²
$Height_i$	第 i 个孕妇的身高	cm
$Weight_i$	第 i 个孕妇的体重	kg
Age_i	第 i 个孕妇的年龄	岁
GC_i	第 i 个样本的 GC 含量	%
$GC_{13,i}$	第 i 个样本的 13 号染色体 GC 含量	%
$GC_{18,i}$	第 i 个样本的 18 号染色体 GC 含量	%
$GC_{21,i}$	第 i 个样本的 21 号染色体 GC 含量	%
$Reads_i$	第 i 个样本的唯一比对读段数	个
IVF_i	第 i 个孕妇是否为 IVF 妊娠的指示变量	-
$Parity_i$	第 i 个孕妇的怀孕次数	次
$Health_i$	第 i 个胎儿是否健康的指示变量	-
$Gender_i$	第 i 个胎儿的性别指示变量	-
$Date_i$	第 i 个样本的检测日期	日期
$BloodNum_i$	第 i 个样本的检测抽血次数	次
$TimeDiff_i$	第 i 个样本的采血时间差	天
$PatientID_i$	第 i 个孕妇的代码标识	-
n_j	第 j 个孕妇的检测次数	次
$\bar{Y}_j$	第 j 个孕妇的平均 Y 染色体浓度	%
$Y_{j,k}$	第 j 个孕妇第 k 次检测的 Y 染色体浓度	%
N	总样本数量	个
N_m	男胎样本数量	个
N_f	女胎样本数量	个
P	总孕妇人数	人
k	BMI 分组总数	-
$ G_j $	第 j 个分组的孕妇数量	人
T_j^{opt}	第 j 个分组的最佳 NIPT 检测时点	周
T_{attain}	最早达标孕周	周

五、 数据预处理

原始数据包含男胎检测数据 1082 条记录和女胎检测数据 605 条记录，共计 1687 条临床检测记录，涵盖 31 个原始特征维度。作为医疗样本数据，所有数值均代表真实的临床检测结果。

5.1 缺失值处理

通过缺失值统计分析，发现男胎数据不存在缺失值，女胎数据仅 BMI 数据存在一个缺失值，使用拟合方式进行填充。与男胎数据相比，女胎数据由于生物学特性缺少 Y 染色体相关指标，属于结构性差异而非数据缺失。

5.2 数据类型清洗与标准化

5.2.1 连续变量标准化处理

为消除身高和体重在量纲和数值范围上的差异对后续建模的影响，对孕妇的身高、体重两个基本生理指标采用 Z-score 标准化方法进行归一化处理。标准化变换公式为：

$$Z = \frac{X - \mu}{\sigma} \quad (1)$$

其中 Z 为标准化后的值，X 为原始观测值， μ 为样本均值， σ 为样本标准差。通过标准化处理，使身高、体重数据转换为均值为 0、标准差为 1 的标准正态分布。

5.2.2 分类变量映射处理

针对数据样本中的分类变量，采用数值编码方法将文本类别转换为数值形式。编码方案如下：

- (1) IVF 妊娠：二元分类变量，映射为 {0：自然受孕, 1：IVF 试管婴儿}
- (2) 怀孕次数：离散分类变量，根据频次分布进行有序编码
- (3) 胎儿是否健康：二元分类变量，映射为 {0：否, 1：是}

5.2.3 时间变量数值化

将“孕妇本次检测时的孕周”列从文本格式（如“16w+1”，“16W+1”，“16w”）转换为以周为单位的数值变量。转换规则为：

$$\text{孕周数值} = \text{周数} + \frac{\text{天数}}{7} \quad (2)$$

5.3 数据分析

本节对处理后的数据进行全面分析。为了获得更全面的统计结果，将男胎和女胎的检测数据进行合并处理，通过多维度的可视化图表深入探讨染色体异常检

测、孕妇健康状况与胎儿发育关系、性别差异以及检测技术质量等关键问题，为临床诊断和质量控制提供数据支撑。

5.3.1 孕妇健康状况与胎儿发育关系分析

本小节重点分析孕妇年龄和 BMI 等关键因素对胎儿异常率的影响。图2通过柱状图展示了不同年龄段和 BMI 分组的胎儿异常率分布。

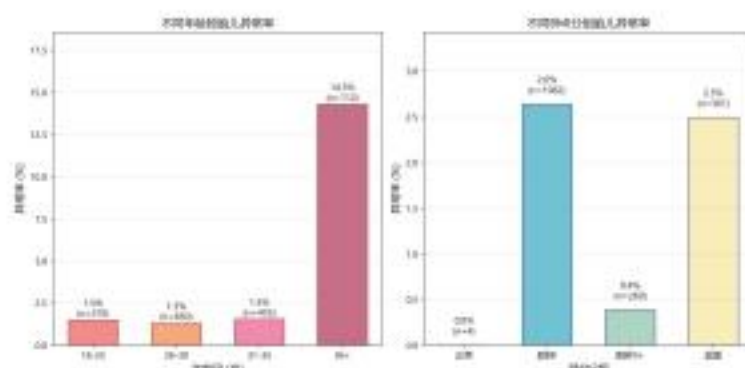


图2 孕妇健康状况与胎儿发育关系分析

从图2可以看出，年龄因素对胎儿异常率有显著影响。18-35 岁组均在 1.5% 附近，而 36 岁以上高龄组异常率显著升高至 14.3%。在 BMI 分组分析中，正常 BMI 组异常率为 0%，肥胖 I 级组异常率为 2.6%，肥胖 II 级以上组为 0.4%，超重组为 2.5%。数据表明，避免肥胖状态可有效降低异常发生率。

5.3.2 染色体 Z 值分布特征分析

本节通过箱线图对各主要染色体的 Z 值分布进行了深入分析。图3展示了 13、18、21 号染色体及 X 染色体在健康与异常胎儿中的 Z 值分布特征，揭示了不同染色体异常检出的统计规律。

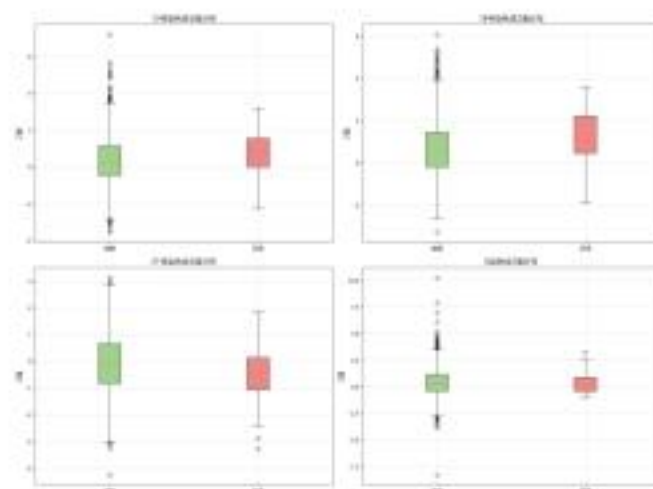


图3 染色体 Z 值分布分析

从图3可知，各染色体 Z 值在健康与异常胎儿间呈现明显差异。18 号染色体异常检出率最高，达 4.68%。13 号染色体异常率为 2.55%，Z 值分布相对集中。21

号染色体异常率最低，仅 0.41%，但其异常样本 Z 值变化幅度较大。X 染色体异常率为 3.08%，在性别鉴定中起重要作用。

5.3.3 胎儿性别差异分析

本小节重点分析了男女胎儿在孕妇年龄、BMI、身高等关键指标上的分布差异。通过直方图和箱线图的呈现：

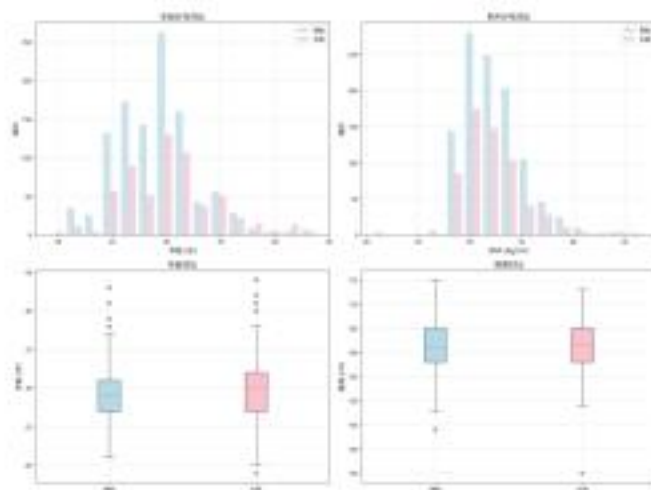


图 4 胎儿性别差异分析

从图4来看，男女胎儿在孕妇年龄分布上基本相似，主要集中在 25-35 岁的最佳生育年龄段。怀男胎和女胎的孕妇 BMI 与身高均值差异并不显著；而年龄箱线图显示怀女胎孕妇年龄分布更为集中，中位数更高，猜测高龄孕妇可能生育女胎的概率更高。

5.3.4 检测技术质量分析

本小节从技术质量角度分析 NIPT 检测的可靠性，重点关注 GC 含量分布、正常率以及各染色体异常检出比例。图5提供了技术质量评估的全方位视角。

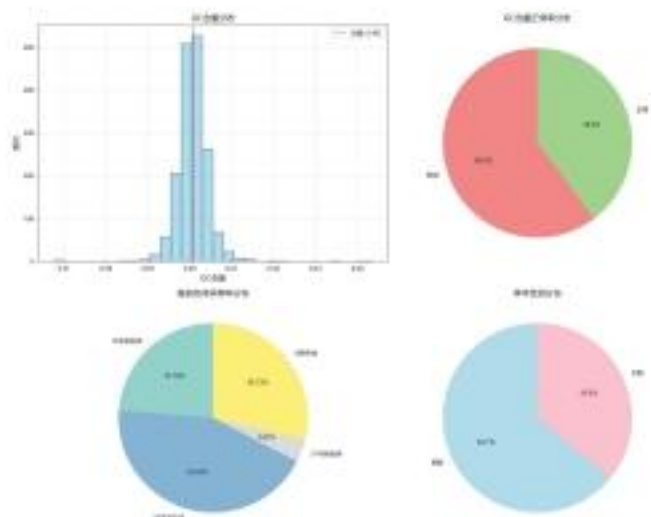


图 5 检测技术质量分析

图5显示，GC 含量基本呈正态分布，表明检测过程的技术稳定性良好。GC 含量正常率为 39.8%，异常率为 60.2%，这一比例在 NIPT 检测中属于正常范围，反映了测序质量控制的有效性。在染色体异常分布中，18 号染色体异常占比最高，其次是 X 染色体、13 号染色体和 21 号染色体。

六、 问题一的模型建立与求解

6.1 相关性分析

为确保模型输入的准确性和后续模型求解的可行性，在建立 NIPT 的时点选择与胎儿的异常判定模型前，需要对各关键变量进行全面的相关性分析。本节通过多维度的统计分析，深入探讨 Y 染色体浓度与各影响因素之间的关系模式。

首先，基于男胎检测的 1082 个样本的统计分析显示：Y 染色体浓度呈右偏分布，数据集中在 0.05-0.15 区间；检测时间呈正态分布，时间跨度 11-29 周；体质质量指数 BMI 呈右偏分布，多数孕妇 BMI 在 28-38 之间，标准化效果良好。

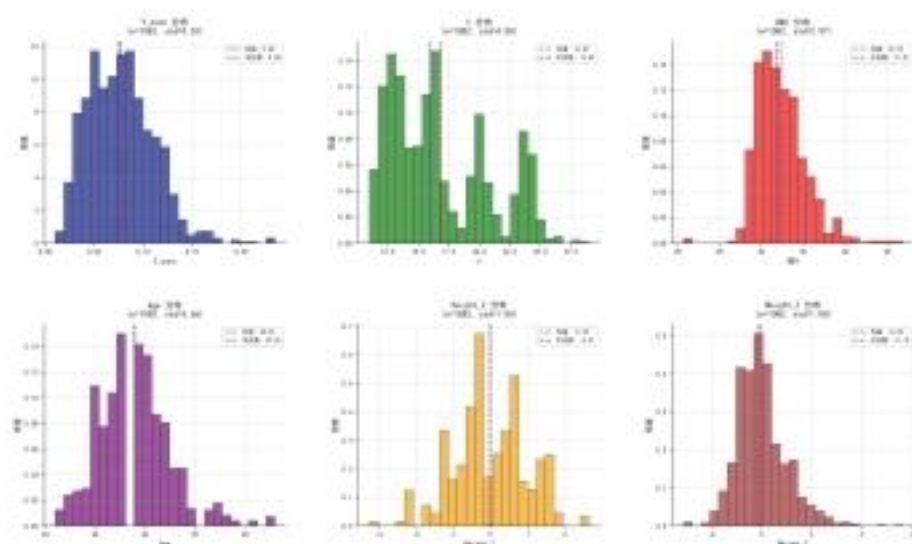


图 6 核心变量分布特征与统计描述

从图6可以看出，数据经过预处理后各变量分布符合预期，均值和方差表现稳定。在此基础上，进一步对核心变量间的相关性进行深入分析。

结果初步表明：孕周 t 与 Y 染色体浓度呈显著正相关，BMI 与 Y 染色体浓度呈负相关，标准化身高 Height_Z 与 Y 染色体浓度呈负相关；同时大多数数据样本均显示随着检测抽血次数 BloodNum 的增加，Y 染色体浓度也会呈现上升趋势，为问题二中确认最早达标孕周的三阶段分层决策提供支撑。

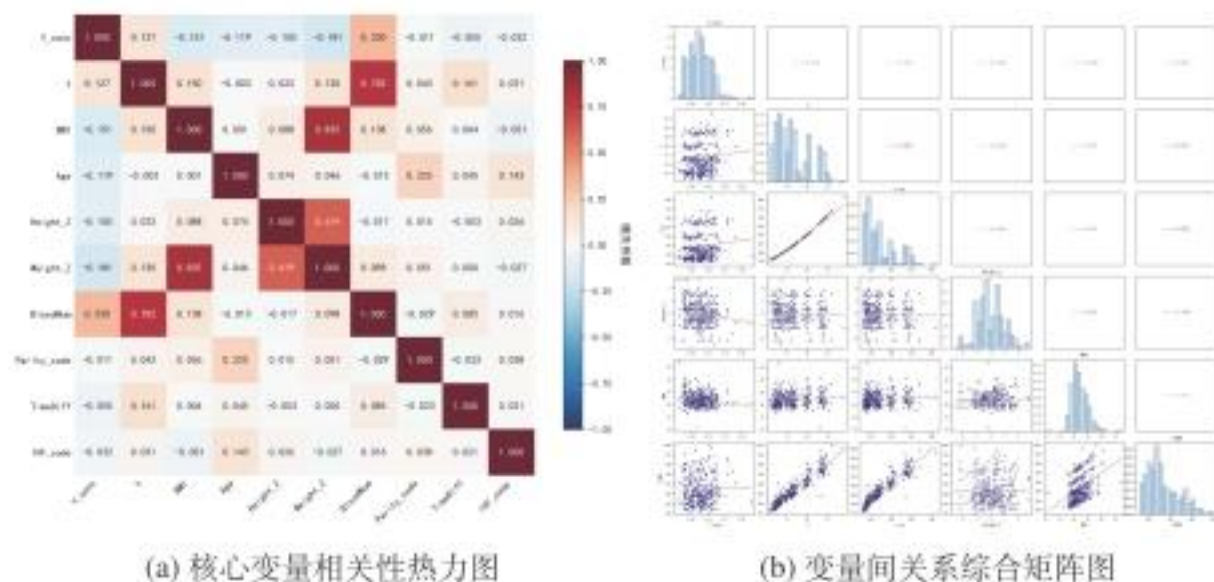
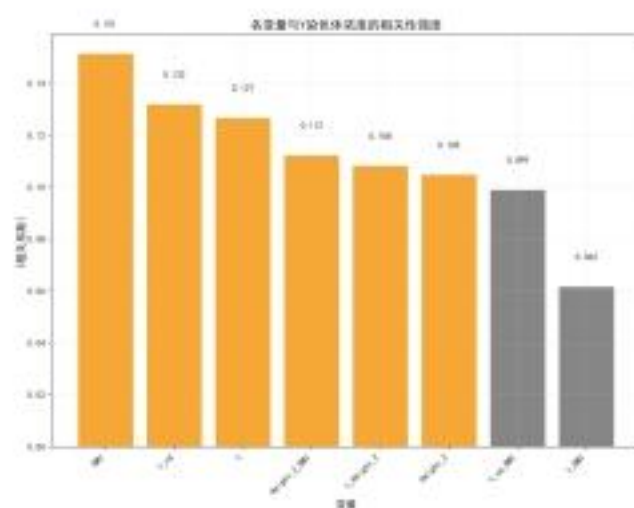


图7 变量相关性分析

散点图矩阵在原先基础上，进一步增加高次项，发现孕周二次项 t^2 与 Y 染色体浓度存在显著正相关性，表明时间效应可能存在非线性特征。同时，BMI 与 Weight_Z 之间的强相关性，使得后续模型建立时，需要谨慎考虑变量间的多重共线性问题。



最后，将各变量与 Y 染色体浓度的相关性强度展示如上图8。其中，不难看出，针对于交互项对 Y 染色体浓度的影响程度存在显著差异。该结果为下一小节模型设定提供一定的参考价值。

6.2 关于线性混合效应模型的设定

在样本数据观测值中分为两个层次，第一层为检测记录层，第二层为孕妇个体层。具体表现为：同一孕妇可能存在多次采血检测或一次采血多次检测的情况。因此，形成了多个检测记录嵌套在同一个孕妇个体内的数据结构。这种数据结构

会引发两个问题：第一，同一孕妇的多次 Y 染色体浓度测量值之间必然存在相关性，违背了传统线性回归中观测值独立性的基本假设；第二，不同孕妇由于个体差异可能存在系统性的基线差异，即个体间异质性。

线性混合效应模型（linear mixed model, LMM）是专门用于处理此类分层数据的统计方法，该模型通过引入随机效应项来捕捉个体层面的随机变异，同时保留固定效应项来估计总体层面的平均效应，从而能够同时处理个体内相关性和个体间异质性问题，确保统计推断的有效性和准确性。其基本形式为：

$$Y_{ij} = X_{ij}\beta + Z_{ij}b_i + \epsilon_{ij} \quad (1)$$

其中 Y_{ij} 为第 i 个个体第 j 次观测值， $X_{ij}\beta$ 为固定效应， $Z_{ij}b_i$ 为随机效应（ $b_i \sim N(0, D)$ ）， $\epsilon_{ij} \sim N(0, \sigma^2)$ 为随机误差。

Step 1: 构造 Y 染色体浓度与孕周数和 BMI 的基准模型

从已有文献和相关性分析不难看出，Y 染色体浓度与孕周及 BMI 存在紧密联系。基于生理机制分析，高 BMI 孕妇因母体血容量增大可能导致胎儿 DNA 稀释，从而减弱孕周效应，故引入 $t \times BMI$ 交互项；同时考虑到可能存在的非线性关系，引入 t^2 和 BMI^2 二次项，初始模型设定如下：

$$Y_{ij} = \beta_0 + \beta_1 \times t_{ij} + \beta_2 \times BMI_{ij} + \beta_3 \times t_{ij}^2 + \beta_4 \times BMI_{ij}^2 + \beta_5 \times (t_{ij} \times BMI_{ij}) + \mu_j + \epsilon_{ij} \quad (2)$$

通过逐步添加各项并进行显著性检验，对所有可能组合（共 8 种）进行比较。结果显示， t_{ij}^2 项系数 β_3 达到 $P < 0.001$ 显著性水平，纳入最终模型；而 BMI_{ij}^2 项和交互项 $(t_{ij} \times BMI_{ij})$ 系数均未达到 $P < 0.05$ 显著性水平。根据奥卡姆剃刀原理，剔除不显著项以避免过度参数化。因此，将 BMI_{ij}^2 项和交互项 $(t_{ij} \times BMI_{ij})$ 从模型中删除。

Step 2: 构造纳入其他指标的扩展模型

为了进一步探索可能影响 Y 染色体浓度的其他因素，采用逐步回归的方法，对孕妇的其他生理指标进行显著性检验。通过统计分析发现，身高的一次项达到显著性水平，因此将其纳入最终模型。经过变量筛选和模型优化后，得到的最终模型结果如下：

$$Y_{ij} = \beta_0 + \beta_1 \times Height_{ij} + \beta_2 \times t_{ij} + \beta_3 \times BMI_{ij} + \beta_4 \times t_{ij}^2 + \mu_j + \epsilon_{ij} \quad (3)$$

6.3 模型求解

本小节采用限制性最大似然估计（REML）方法对线性混合效应模型进行参数估计，该方法在处理不平衡数据和小样本情况下具有更好的统计性质。表1展示了固定效应的详细估计结果。

表1 固定效应估计结果

变量	系数估计	标准误	z 值	p 值
t	-0.003014	0.0013211	-2.28	0.023
t^2	0.0001701	0.0000363	4.68	0.000
$Height_z$	-0.0038334	0.0018705	-2.05	0.040
BMI	-0.0013541	0.0005159	-2.62	0.009
常数项	0.1213106	0.0197865	6.13	0.000

固定效应的联合显著性检验采用 Wald χ^2 检验, 统计量为 $\chi^2(4) = 440.90, p < 0.001$, 在 0.001 的显著性水平下拒绝所有固定效应系数均为零的原假设, 表明至少有一个预测变量对 Y 染色体浓度具有显著影响。

(1) 孕周非线性效应分析

孕周二次项 (t^2) 系数为 0.0001701, 线性项 (t) 系数为 -0.003014, 分别在 99% 和 95% 水平下显著。正的二次项系数与负的线性项系数形成 U 型非线性关系, 理论转折点为 $t_{min} = -\frac{\beta_1}{2\beta_2}$ 。由于观测孕周均在 10 周以上, 观测到 U 型曲线右侧递增部分, 表现为 Y 染色体浓度随孕周加速增长且增长的速度越来越快。

(2) 孕妇个体特征效应分析

标准化身高 ($Height_z$) 在 99% 水平下显著, 回归系数表明身高每增加 1 个标准差, Y 染色体浓度降低 0.38334%, 可能与血液稀释效应相关。

BMI 在 99% 水平下显著, 其系数为 -0.0013541, 表明 BMI 每增加 1 kg/m², Y 染色体浓度降低 0.13541%。该负向关系反映 BMI 较高孕妇因血容量增加导致 Y 染色体浓度相对稀释。

6.4 模型检验

6.4.1 随机效应必要性检验

随机效应的必要性检验是验证线性混合效应模型相对于普通线性回归模型优势的关键步骤。我们通过多种方法评估了在模型中纳入随机截距的必要性。

(1) 模型比较结果

表2展示了线性混合效应模型与固定效应模型的详细比较结果。

表2 混合效应模型与固定效应模型比较

指标	混合效应模型	固定效应模型
样本量	1,082	1,082
对数似然值	2547.5722	2171.4381
AIC	-5081.1444	-4330.8762
BIC	-5046.2385	-4300.9568
参数个数	7	5

从模型比较结果可以看出，混合效应模型在所有拟合指标上均显著优于固定效应模型。对数似然值从 2171.4381 提升至 2547.5722，增幅达 376.1341，表明混合效应模型对数据的拟合能力大幅改善。此外，尽管混合效应模型增加了 2 个参数，但其 AIC 值和 BIC 值均远低于固定效应模型，表明考虑惩罚项后，混合效应模型仍具有明显优势。

(2) 似然比检验

混合效应模型内置的似然比检验结果显示： $\chi^2(1) = 752.27, p < 0.001$ ，强烈支持随机截距的必要性。该检验统计量极大且 p 值小于 0.001，表明随机效应方差显著不为零，拒绝了“无需随机效应”的零假设。

(3) 方差成分分析

随机截距方差 $\sigma_u^2 = 8.24 \times 10^{-4}$ ，残差方差 $\sigma_e^2 = 2.85 \times 10^{-4}$ 。组内相关系数为：

$$ICC = \frac{\sigma_u^2}{\sigma_u^2 + \sigma_e^2} = \frac{0.000824}{0.000824 + 0.000285} = 0.743 \quad (4)$$

ICC = 0.743 表明约 74.3% 的总变异来源于孕妇间差异，同一孕妇内部的重复测量之间存在强烈的相关性，约四分之三的总体变异来源于孕妇间的个体差异。该结果证明了数据的聚类特征，使用普通线性回归会严重低估标准误，导致统计推断出现错误。

6.4.2 随机截距正态性检验

混合效应模型的一个重要假设是随机效应服从正态分布。为验证随机截距的正态性假设，本研究采用 Q-Q 图和直方图两种方法进行诊断。

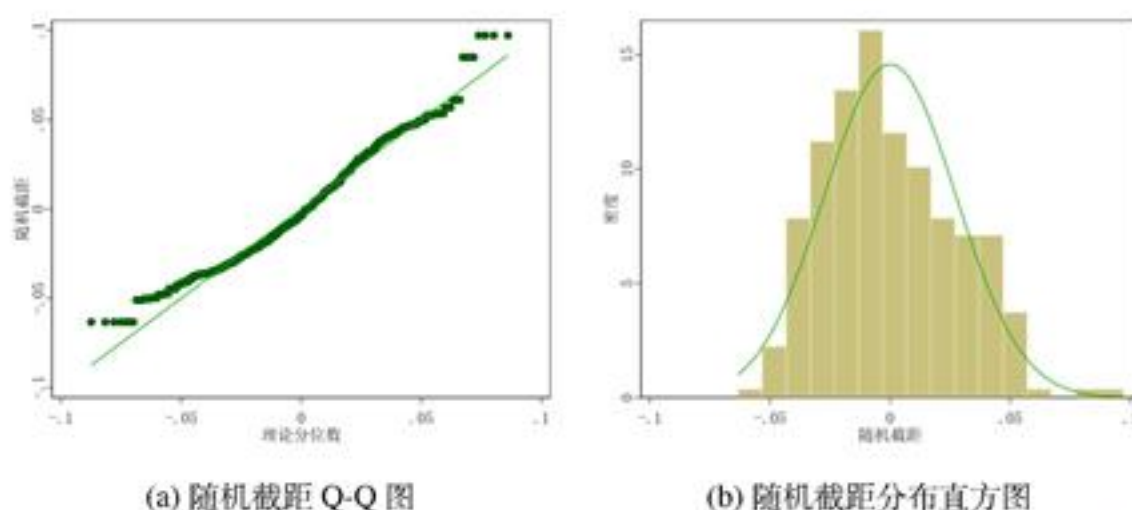


图 9 随机截距正态性检验

从图9的诊断结果可以看出，随机截距基本满足正态分布假设。Q-Q 图显示大部分观测点紧密分布在理论直线附近，仅在右尾部分存在轻微偏离；分布直方图呈现良好的钟形特征，与理论正态曲线高度吻合，分布中心接近零且基本对称。

这些结果表明混合效应模型中随机截距的正态性假设得到较好满足，为模型推断的有效性提供了支撑。

6.4.3 同方差性检验

同方差性是线性混合效应模型的重要假设之一，要求残差的方差在不同拟合值水平上保持相等。我们通过多角度的图形分析对这一假设进行了全面检验。

图10展示了四种不同类型的残差与拟合值关系图：

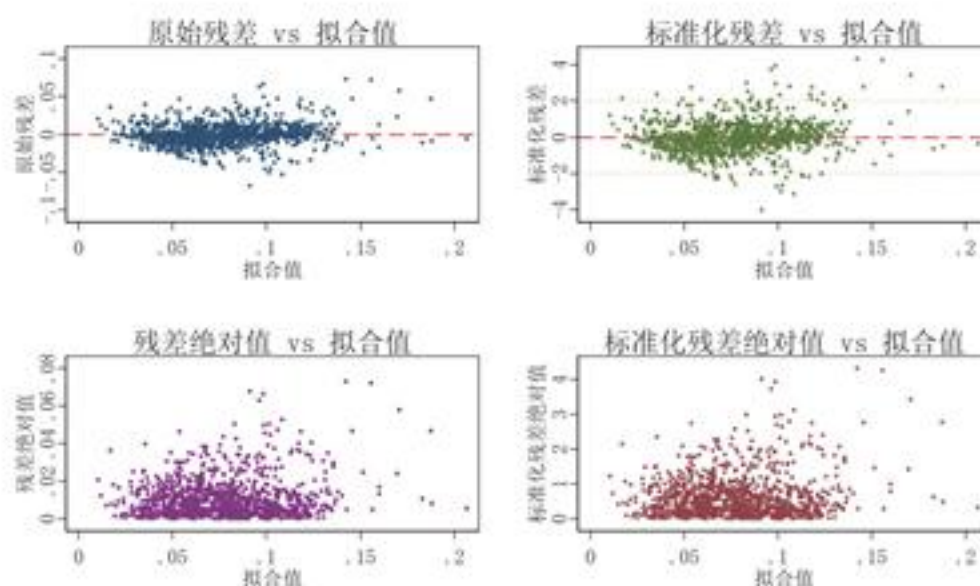


图 10 同方差性检验

上图表明，原始残差围绕零线随机分布，且集中在 ± 0.05 范围内，标准化残差大部分位于 ± 2 标准差内，残差绝对值无明显递增或递减趋势。因此，残差分布基本符合同方差性假设要求，异常值呈随机分布，支持同方差性假设。

图11展示了带有 LOWESS 平滑曲线的残差分析，用于检验系统性偏差和非线性模式：

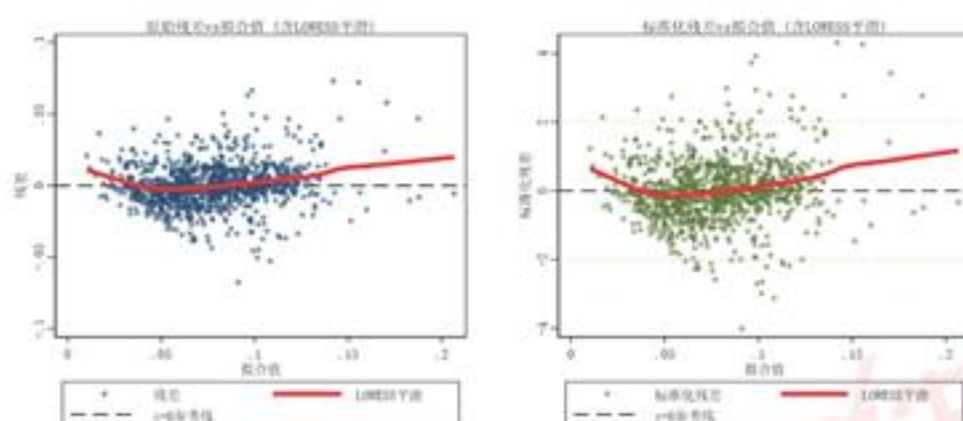


图 11 LOWESS 平滑分析

LOWESS 平滑分析显示，原始残差和标准化残差的平滑曲线均在零线附近小

幅波动,虽然存在轻微的 U 型模式,但偏离程度很小,表明当前的二次项设定基本充分,无明显的模型设定偏误,系统性偏差可忽略。

综合上述诊断结果,模型残差分布基本符合同方差性假设,结合线性混合效应模型的稳健性特征,当前模型的同方差性假设得到较好支持。

七、 问题二的模型建立与求解

问题二试对男胎孕妇的 BMI 进行合理分组,并求出对应的 BMI 区间和最佳 NIPT 时点,使得孕妇可能的潜在风险最小。在问题一中可以探索到胎儿 Y 染色体浓度与孕期存在正相关关系,因此“晚检测”可能有利于减小检验的误差。然而,实际中应尽早发现不健康的胎儿,否则会带来治疗窗口期缩短的风险,这就需要进行数学建模探索最佳的 NIPT 时点。

7.1 数据聚合

可以观察到原始数据的颗粒度为“单次检测记录”,每一行代表一个孕妇在特定时间点的检测结果。然而,“单次检测记录”只能反映孕妇的检查时间和孕期,并不能体现每个孕妇的“最早达标时间”。为了保证数据质量和准确,本问题的分析单位应当是“单个孕妇”,因此需要进行数据聚合转换。

首先,从原始数据集中筛选出所有男胎孕妇的检测记录,识别出 267 位独立的孕妇个体。对于每位孕妇,执行以下聚合操作:

- (1) **BMI 计算**: 计算该孕妇在整个检测期间的平均 BMI 值,作为其个体特征。
- (2) **序列构建**: 整理出该孕妇的所有检测记录,构成包含 (孕周, Y 染色体浓度) 的观测序列。

经数据聚合,最终构建了 267 例孕妇的完整数据,每一行均包含平均 BMI、总检测次数及完整的 (孕周, Y 染色体浓度) 观测时间序列等核心信息,以便为后续建模分析。

7.2 最早达标孕周 T_{attain} 预测模型: 改良的高斯过程回归 (GPR)

为了优化分组策略,首先需要为每一位孕妇构建一个核心变量: 最早达标孕周 (T_{attain}), 即其 Y 染色体浓度达到 4% 阈值的最早预测时间。

由于每位孕妇的观测点非常稀疏,且 Y 染色体浓度的增长趋势具有非线性特征,传统的线性回归模型误差较大。此处选用了高斯过程回归 (Gaussian Process Regression, GPR) 进行回归预测。

7.2.1 GPR 原理与适用性

高斯过程回归作为一种非参数贝叶斯方法,其核心思想是将待估计的函数 $f(t)$ 本身视为一个随机过程。设 Y 染色体浓度随孕周的变化函数为 $f(t)$, 其服从高斯

过程:

$$f(t) \sim \mathcal{GP}(m(t), k(t, t')) \quad (5)$$

核函数选择: 采用径向基函数核捕捉平滑增长特性

$$k(t, t') = \sigma_f^2 \exp\left(-\frac{(t - t')^2}{2\ell^2}\right) + \sigma_n^2 \delta_{t,t'} \quad (6)$$

预测分布: 给定观测数据 $\mathcal{D}_i$, 新孕周 t_* 的预测为

$$y_* | \mathcal{D}_i \sim \mathcal{N}(\mu_*(t_*), \sigma_*^2(t_*)) \quad (7)$$

预测公式: 均值和方差计算

$$\mu_*(t_*) = \mathbf{k}_*^T (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y} \quad (8)$$

$$\sigma_*^2(t_*) = k(t_*, t_*) - \mathbf{k}_*^T (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_* \quad (9)$$

GPR 方法的关键优势在于其能够量化预测不确定性, 适用于临床决策。尤其当观测点较少时, GPR 会自动放大预测不确定性, 从而能够避免了过度自信的预测。

7.2.2 三阶段分层决策流程

每位孕妇的 T_{attain} 值通过三阶段分层决策流程确定。由于数据稀疏和临床要求, 预测遵循“真实数据优先于模型预测, 保守预测优先于无效判定”的原则。决策流程涵盖: 预处理与个体分类、核心估计量计算、基于数据可靠性的终末修正, 实现对最早达标孕周的严谨推断。

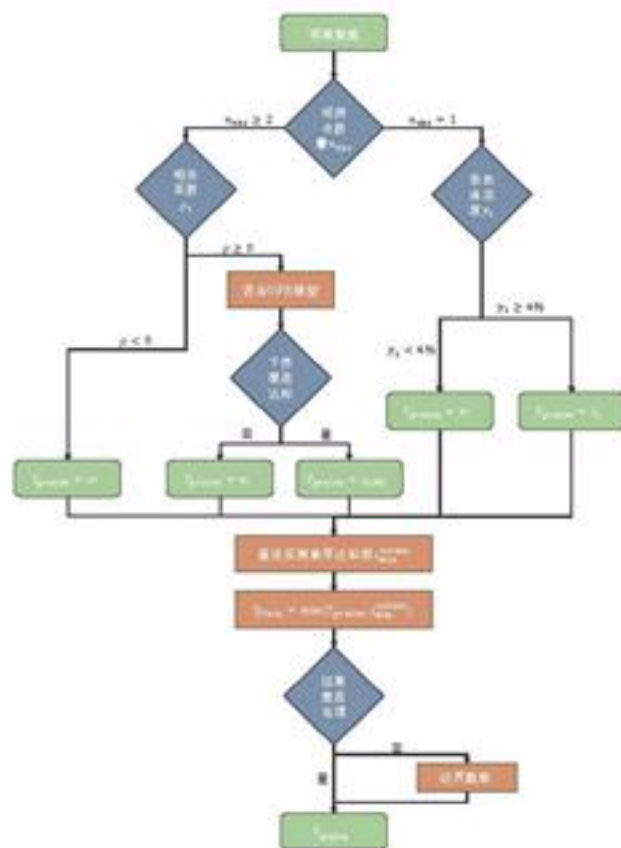


图 12 三阶段分层决策流程图

阶段一：预处理与个体分类

对于每一位孕妇，我们首先获取其完整的（孕周，Y 染色体浓度）观测序列，并根据序列的长度进行初步分类：观测点数量 $n_{obs} = 1$ 时进入“单点观测”处理分支； $n_{obs} \geq 2$ 时进入“多点观测”处理分支。

阶段二：核心估计量计算

(1) 单点观测处理

当仅有一个观测点，应检查该点的浓度值 y_1 ：若 $y_1 \geq 4\%$ ，该孕妇在观测时点 t_1 已明确达标，初步结果为 $T_{attain} = t_1$ ；若 $y_1 < 4\%$ ，由于缺乏趋势信息无法进行外推，初步结果为 $T_{attain} = \infty$ 。

(2) 多点观测处理

对具备多次观测的孕妇，首先计算孕周序列 $t = (t_1, \dots, t_n)$ 与对应 Y 染色体浓度序列 $y = (y_1, \dots, y_n)$ 的皮尔逊相关系数

$$\rho_{ty} = \text{corr}(t, y). \quad (10)$$

若 $\text{corr}(t, y) < 0$ ，这表明观测数据显示浓度随孕周增加而下降，与问题一所得的常规结论相悖。在此条件下，暂时无法建立有意义的增长模型，初步结果为 $T_{attain} = \infty$ ，并将在阶段三结合全局信息进行最终决策。

若 $\text{corr}(t, y) \geq 0$ ，趋势合理，则以 (t, y) 拟合独立 GPR 模型，在 10–40 周区间预测 $(\mu(t), \sigma(t))$ ，并计算 95% 置信下界 $\mu(t) - 1.96\sigma(t)$ 。若存在首次使下界 $\geq 4\%$

的孕周 t_{gpr} , 则令 $T_{attain} = t_{gpr}$; 若至 40 周仍无满足点, 或 GPR 拟合失败, 则 $T_{attain} = \infty$ 。

阶段三：基于数据可靠性的终末修正

(1) 实测达标点核查

对该孕妇全部观测序列 $\{(t_i, y_i)\}$, 提取所有满足 $y_i \geq 4\%$ 的实测点, 记

$$t_{actual}^{\min} = \min\{t_i \mid y_i \geq 0.04\}. \quad (11)$$

若集合为空, 则 $t_{actual}^{\min} = +\infty$ 。

(2) 结果修正规则

比较 $t_{actual}^{\min}$ 与阶段二初步 $T_{attain}^{\text{prelim}}$:

$$T_{attain} = \begin{cases} t_{actual}^{\min}, & t_{actual}^{\min} < T_{attain}^{\text{prelim}}, \\ T_{attain}^{\text{prelim}}, & \text{else.} \end{cases} \quad (12)$$

其中 $T_{attain} = t_{actual}^{\min}$ 当且仅当 $T_{attain}^{\text{prelim}} = \infty$ 时亦成立。

(3) 边界截断

对所有有限 T_{attain} , 若 $T_{attain} < 10$ 周, 则强制修正为

$$T_{attain} = 10. \quad (13)$$

通过三阶段分层决策, 使得每位孕妇获得唯一且逻辑严谨的 T_{attain} 。

7.3 模型建立

传统 NIPT 检测采用统一时点策略, 未考虑孕妇个体差异, 可能影响检测精度。临床研究显示, 孕妇 BMI 指数直接影响胎儿 Y 染色体浓度达标时间。据此建立基于 BMI 分层的个体化 NIPT 时点推荐体系。

本小节将 BMI 分组建模为多目标约束优化。决策变量包括 BMI 分组边界 $\{b_1, b_2, \dots, b_{k-1}\}$ 和各组最佳检测时点 $\{t_1, t_2, \dots, t_k\}$ 。对于 BMI 范围 $[b_{j-1}, b_j)$ 的分组 G_j , 在确保检测最多具有 (1-P) 误差下, 优化检测时点 t_j 以最小化该组综合风险。

基于此, 本小节设置如下决策变量:

(1) BMI 分组数量 k

该变量为正整数决策变量, 表示将男胎孕妇群体按 BMI 指标划分的分组总数。基于临床实践的考虑, 确保既能体现不同 BMI 水平孕妇的个体化差异, 又需要避免过度细分。

(2) BMI 分割点集合 $B = \{b_1, b_2, \dots, b_{k-1}\}$

该变量集合为连续型决策变量, 表示将孕妇 BMI 连续分布划分为 k 个互不重叠区间的 $k-1$ 个分界点。其中 b_i 表示第 i 个分割点的 BMI 数值, 需满足严格递增约束条件 $b_1 < b_2 < \dots < b_{k-1}$, 且所有分割点必须位于观测数据的有效范围内, 从而构成了 k 个 BMI 分组区间。

(3) 各组最佳 NIPT 检测时点 T_j^{opt}

该变量为连续型决策变量, 表示第 j 个 BMI 分组 (G_j) 内孕妇的推荐 NIPT 检测孕周。 T_j^{opt} 基于该组内所有孕妇 Y 染色体浓度达标时间 T_{attain} 的统计分布确定。

7.3.1 约束条件的设置

(1) BMI 分割点有序性约束

该约束条件用于确保 BMI 分组边界的合理性, 即所有 BMI 分割点必须严格按照数值大小递增排列, 形成互不重叠的连续区间。即

$$\begin{aligned} BMI_{min} < b_1 < b_2 < \dots < b_{k-1} < BMI_{max}, \\ b_i &\in [BMI_{min} + \delta, BMI_{max} - \delta], \forall i \in \{1, 2, \dots, k-1\} \end{aligned} \quad (14)$$

其中, BMI_{min} 和 BMI_{max} 分别表示数据集中孕妇 BMI 的最小值和最大值, δ 为边界缓冲参数。

(2) NIPT 时点计算约束

该约束保证了检测误差的可控性, 它将“检测准确率”这一业务指标转化为具体的数学定义。对于给定的准确率 α , 每个分组的最佳 NIPT 时点, 被定义为该组所有孕妇 T_{attain} 数据分布的第百分位数。

$$T_j^{opt} = \inf \left\{ t \mid \frac{| \{ i \in G_j \mid t_i \leq t \} |}{|G_j|} \geq \alpha \right\}, \quad \forall j \in \{1, 2, \dots, k\} \quad (15)$$

其中, G_j 代表第 j 个分组的孕妇集合, $|G_j|$ 表示该 BMI 分组内的孕妇人数, t_i 为该组内孕妇 i 的达标孕周。此公式确保了在推荐的 T_j^{opt} 时点进行检测, 组内至少有比例为 α 的孕妇已经达标。

(3) 最小组规模统计约束

该约束条件用于保证每个 BMI 分组具有充分的样本量以避免因过度细分导致的小样本偏差。具体而言则是每个分组内的孕妇人数不得少于预设的最小阈值。即

$$N_{min} = \left\lfloor \frac{N}{k} \cdot 0.6 \right\rfloor \quad (16)$$

这里 N 是总样本数。该设定保证了最小样本数是平均组规模的 60%, 在分组的灵活性和稳定性之间取得了更好的平衡。

(4) 孕妇分组分配约束

这是一个基础性约束, 它明确了孕妇个体如何根据其 BMI 值被分配到唯一的分组中。为方便定义, 我们设 $b_0 = BMI_{min}$ 及 $b_k = BMI_{max}$ 。

$$G_j = \{ i \mid b_{j-1} < BMI_i \leq b_j \}, \quad \forall j \in \{1, 2, \dots, k\} \quad (17)$$

该约束确保了所有孕妇都被划分完毕, 且每位孕妇只属于一个分组。

7.3.2 目标函数的设置

考虑到不同 BMI 分组策略对整体临床风险的影响，综合潜在风险最小化目标函数设置为：

$$\min_{k,B} \text{TotalRisk} = \sum_{j=1}^k |G_j| \cdot \text{Risk}(T_j^{\text{opt}}(P)) \quad (18)$$

其中， k 表示 BMI 分组总数， $B = \{b_1, b_2, \dots, b_{k-1}\}$ 表示 BMI 分割点集合， $|G_j|$ 表示第 j 个分组的孕妇人数， $T_j^{\text{opt}}(P)$ 表示第 j 组在百分位数 P 下的最佳 NIPT 检测时点， $\text{Risk}(T_j^{\text{opt}}(P))$ 为基于检测时点的风险量化函数。

本目标函数综合考虑了不同分组策略下的风险分布差异，其中 $\sum_{j=1}^k |G_j| \cdot \text{Risk}(T_j^{\text{opt}}(P))$ ，表示各分组总风险。风险函数 $\text{Risk}(t)$ 按检测时点分级赋权：早期检测 ($t \leq 12$ 周) 低权重，中期检测 ($12 < t \leq 27$ 周) 中等权重，晚期检测 ($t > 27$ 周) 高权重。通过最小化风险加权和，实现 NIPT 策略的风险最优配置。

7.3.3 多约束下风险最小化模型的建立

结合上述约束条件与设置的目标函数，建立了基于 BMI 分组的男胎孕妇最佳 NIPT 时点决策模型：

$$\begin{aligned} \min_{k,B} \text{TotalRisk} &= \sum_{j=1}^k |G_j| \cdot \text{Risk}(T_j^{\text{opt}}(P)) \quad (19) \\ \text{s.t.} \quad &\begin{cases} BMI_{\min} < b_1 < b_2 < \dots < b_{k-1} < BMI_{\max}, & b_i \in [BMI_{\min} + \delta, BMI_{\max} - \delta] \\ & \forall i \in \{1, 2, \dots, k-1\} \\ T_j^{\text{opt}} = \inf \left\{ t \mid \frac{|\{i \in G_j \mid t_i \leq t\}|}{|G_j|} \geq \alpha \right\}, & \forall j \in \{1, 2, \dots, k\} \\ N_{\min} = \left\lfloor \frac{N}{k} \cdot 0.6 \right\rfloor \\ G_j = \{i \mid b_{j-1} \leq BMI_i < b_j\}, & \forall j \in \{1, 2, \dots, k\} \end{cases} \end{aligned}$$

7.4 基于遗传算法的 BMI 分组与 NIPT 时点风险最小化问题

本研究构建的 BMI 分组与 NIPT 时点联合优化问题具有变量多、目标函数非连续、非凸的特点，属于典型的 NP-hard 组合优化问题。传统的梯度优化方法难以有效处理此类问题，因此我们采用遗传算法 (Genetic Algorithm, GA) 进行求解。

遗传算法作为经典的进化计算方法，具有较强的全局搜索能力和跳出局部最优的能力，契合本问题的特征。

7.4.1 算法设计与实现

Step 1: 编码策略

针对 BMI 分组边界的连续性特点，采用实数编码方式。每个个体表示为一个 $k-1$ 维向量 $\mathbf{B} = [b_1, b_2, \dots, b_{k-1}]$ ，其中 b_i 表示第 i 个 BMI 分割点，满足 $BMI_{min} \leq b_1 < b_2 < \dots < b_{k-1} \leq BMI_{max}$ 的有序约束。

Step 2: 适应度函数

适应度函数设计为风险最小化的倒数形式，以处理约束违反情况：

$$\text{Fitness}(\mathbf{B}) = \frac{1}{\text{TotalRisk}(\mathbf{B}) + \text{Penalty}(\mathbf{B}) + \epsilon} \quad (20)$$

其中 ϵ 为避免除零的小常数，惩罚项 $\text{Penalty}(\mathbf{B})$ 针对违反动态最小组规模约束的情况：

$$\text{Penalty}(\mathbf{B}) = \lambda \sum_{j=1}^k \max\left(0, \left\lfloor \frac{N_{total}}{k} \cdot \alpha \right\rfloor - |G_j|\right)^2 \quad (21)$$

其中 λ 为惩罚系数， $\alpha = 0.6$ 为比例因子， $|G_j|$ 为第 j 组的实际人数。

Step 3: 遗传操作

(1) **选择操作**：采用锦标赛选择策略，从种群中随机选择 s 个个体进行竞争，选出适应度最高者进入交配池，兼顾选择压力与种群多样性。

(2) **交叉操作**：使用改进的均匀交叉算子 (Uniform Crossover)，对于父代个体 $\mathbf{B}_1$ 和 $\mathbf{B}_2$ ，子代个体 $\mathbf{B}_c$ 的第 i 个基因按概率 p_c 从父代中选择：

$$B_{c,i} = \begin{cases} B_{1,i}, & \text{if rand}(0, 1) < p_c \\ B_{2,i}, & \text{otherwise} \end{cases} \quad (22)$$

交叉后需对分割点进行排序以维持有序约束。

(3) **变异操作**：采用定制化的高斯变异算子，对选中的基因添加正态分布扰动：

$$B_{m,i} = B_i + \mathcal{N}(0, \sigma^2 \cdot (BMI_{max} - BMI_{min})) \quad (23)$$

其中 σ 为变异强度参数，随迭代次数自适应调整： $\sigma = \sigma_0 \cdot \exp(-\beta \cdot t / T_{max})$ 。

Step 4: 约束处理机制

为确保生成的分割点满足有序性和边界约束，在每次遗传操作后进行约束修复：

(1) **边界修复**：将超出 $[BMI_{min}, BMI_{max}]$ 范围的分割点截断到边界值

(2) **有序修复**：对分割点向量进行升序排列

(3) **最小间距保证**：确保相邻分割点间距不小于预设阈值 δ_{min}

Step 5: 算法参数设置

基于问题特点和实验调优，算法参数设置为：种群规模 $N_p = 100$ ，最大迭代次数 $T_{max} = 500$ ，交叉概率 $p_c = 0.8$ ，变异概率 $p_m = 0.1$ ，锦标赛规模 $s = 3$ ，惩罚系数 $\lambda = 1000$ 。

7.4.2 算法收敛性与稳定性

为保证算法的收敛性和解的稳定性，采用以下策略：

(1) **精英保留**：每代保留适应度最高的 $[0.1 \cdot N_p]$ 个个体直接进入下一代，防止最优解丢失。

(2) **多样性维护**：通过监控种群多样性指标，当种群过早收敛时增加变异概率或引入新的随机个体。

(3) **多次独立运行**：对每个分组数 k 和准确率场景进行 10 次独立运行，取最优结果作为最终解，提高解的可靠性。

该遗传算法的设计充分考虑了问题的特殊性，通过定制化的编码、交叉、变异策略和约束处理机制，能够有效求解 BMI 分组与 NIPT 时点的联合优化问题，为临床决策提供科学依据。

最终，基于遗传算法的流程如图所示：



图 13 遗传算法流程图

7.4.3 最优分组数 k 的探索

分组数的选择需要在精细化管理和策略稳定性之间找到平衡。我们对 $k \in \{3, 4, 5, 6\}$ 的所有情况进行完整的优化计算，其中 $k = 3$ 体现基本分层思想， $k = 6$ 时每组平均样本量约为 43 例，满足统计分析要求。

对于每个准确率场景 $P \in \{50\%, 75\%, 90\%, 95\%, 99\%\}$ ，第 j 组的推荐时点为该组内 T_{attain} 值的第 P 百分位数：

$$t_j(P) = \text{Percentile}(\{T_{attain,i} \mid i \in G_j\}, P) \quad (24)$$

优化目标函数为：

$$\min_B \text{TotalRisk}(k, P) = \sum_{j=1}^k |G_j| \cdot \text{Risk}(t_j(P)) \quad (25)$$

每个 (k, P) 组合进行 10 次独立的遗传算法运行，最优分组数选择准则为：

$$k^*(P) = \arg \min_{k \in \{3,4,5,6\}} \text{Mean}(k, P) \quad (26)$$

通过比较不同 k 值的最终总风险，我们为每个准确率场景选定风险最小的分组数作为最优策略，结果显示 $k = 3$ 在所有准确率场景下均能实现最低总风险值，证明了三分组策略在精细化管理与策略稳定性之间达到了最佳平衡。

7.5 不同检测误差下的最优分组策略

模型针对不同检测误差下，即在不同的 NIPT 检测有效性的情况下，可以计算得出不同的 BMI 分组与 NIPT 时点推荐方案。

表 3 50% 准确率的早期筛查策略

BMI 分组	组内人数	建议 NIPT 时点 (孕周)	风险等级
(, 29.88]	55	11.00	低
(29.89, 31.82]	82	11.64	低
(31.82,)	125	12.00	低

在 50% 准确率要求下，将 260 名男胎孕妇分为三组，BMI 分界点分别为 29.88 和 31.82 kg/m^2 ，对应的 NIPT 推荐时点为 11.00-12.00 孕周。这一发现表明，通过适度放宽首次检测的准确率要求，可以将所有孕妇的潜在风险控制在最低水平。

表 4 75% 准确率的进取平衡策略

BMI 分组	组内人数	建议 NIPT 时点 (孕周)	风险等级
(, 29.89]	54	12.86	高
(29.90, 34.03]	146	13.11	高
[34.09,)	60	16.04	高

当需要更低的检测误差时，即准确率要求从 50% 提升至 75% 时，模型结果显示出显著的风险跃升现象。这一现象的根本原因在于为了保证检测的准确性，推荐时点被迫跨越 12 孕周的临界阈值，触发了风险函数的非连续跳跃。

表 5 90% 准确率的标准平衡策略

BMI 分组	组内人数	建议 NIPT 时点 (孕周)	风险等级
(, 30.33]	68	16.09	高
(30.35, 34.26]	138	15.19	高
(34.27,)	54	20.27	高

表6 99% 准确率的极致保守策略

BMI 分组	组内人数	建议 NIPT 时点 (孕周)	风险等级
(, 31.25]	121	20.71	高
(31.32, 34.16]	83	19.67	高
(34.21,)	56	25.09	高

当要求极小化误差时，即在 75%-99% 准确率区间内，虽然 BMI 分组边界和推荐时点存在差异，但总是在 12-27 周之间，并不会出现在可能出现极高风险的 27 周后。表明在该准确率范围内，模型已达到风险最小化的帕累托前沿。

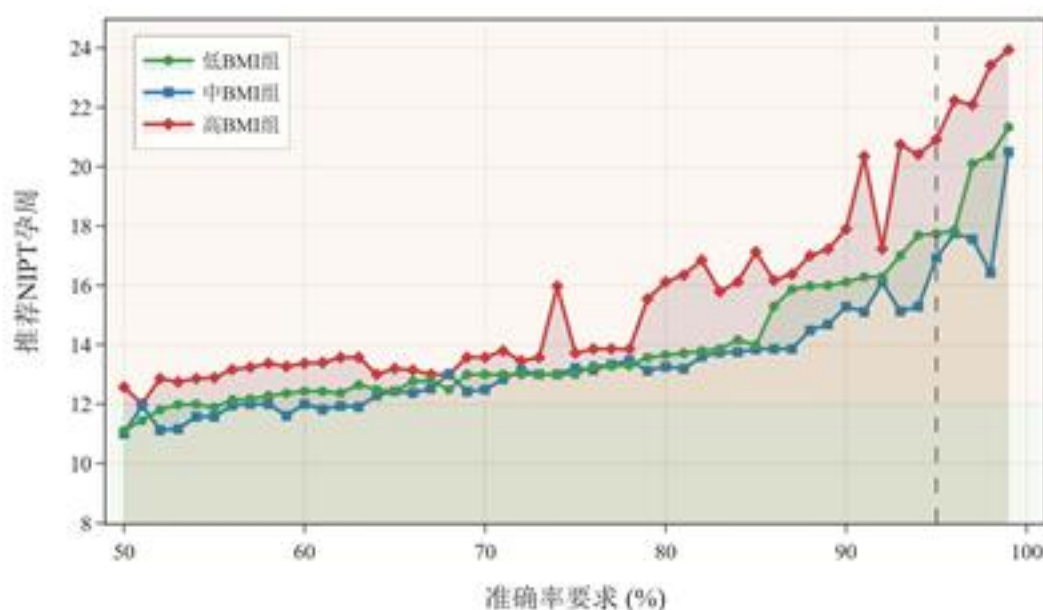


图 14 不同 BMI 组推荐时点的准确率响应曲线

如图14所示，NIPT 策略自适应优化模型的结果清晰展现了三个 BMI 组的推荐时点随准确率要求的变化规律。在 50%-70% 的较高区间内，三组的推荐时点相对接近且变化平缓，均维持在 12-14 孕周的安全窗口内；当准确率要求超过 75% 后，各组时点开始显著分化，高 BMI 组需要更晚的检测时点以确保达标。特别值得关注的是 90% 准确率分界线附近，所有组别的推荐时点均出现快速上升，验证了该准确率水平下风险控制的复杂性。

据此，实验结果推荐各个组别的孕妇的第一次检测尽可能在 12 周前，虽然检测的准确率可能没那么高，但是出于驱离风险的考虑，应当提前进行检测。后续

的检测为了确保准确性，应按高准确率策略推荐的时点进行检测。

八、 问题三的模型建立与求解

男胎 Y 染色体浓度达标时间受孕妇身高、体重、年龄等多种因素影响，且存在显著的个体差异和时序动态特征。为综合考虑这些多维因素并处理检测误差的不确定性，需要构建能够同时处理静态生理特征和动态变化轨迹的优化模型。

8.1 数据转换

本小节旨在将原始的多维数据转换为适合模型训练的标准化格式。

8.1.1 特征构建

基于生存分析的建模范式，本小节构建了两类互补的特征体系：

1. 静态特征向量构建

静态特征向量 $\mathbf{X}_{\text{static}} \in \mathbb{R}^4$ 捕捉每位孕妇的基线生理特征，为模型提供个体差异的先验信息，具体如下：

- (1) 年龄特征：直接提取孕妇的年龄信息，记为 Age_i
- (2) 身高特征：提取孕妇身高，记为 Height_i
- (3) 初始体重特征：选取每位孕妇首次记录的体重值，记为 $\text{Weight}_{i,0}$
- (4) 初始 BMI 特征：选取每位孕妇首次记录的 BMI 值，记为 $\text{BMI}_{i,0}$

静态特征向量的数学表示为：

$$\mathbf{X}_{\text{static},i} = [\text{Age}_i, \text{Height}_i, \text{Weight}_{i,0}, \text{BMI}_{i,0}]^T \quad (27)$$

选择首次记录的体重和 BMI 而非平均值的原因在于：在真实的临床场景中，本文希望在孕妇初次就诊时，就能利用她此刻的信息进行决策。

2. 时序特征序列构建

时序特征序列 $\mathbf{X}_{\text{long},i} \in \mathbb{R}^{L_i \times 2}$ 记录每位孕妇完整的 Y 染色体浓度动态变化轨迹，其中 L_i 为第 i 位孕妇的观测次数。时序特征包含：

- (1) 孕周序列： $\mathbf{t}_i = [t_{i,1}, t_{i,2}, \dots, t_{i,L_i}]^T$
- (2) Y 染色体浓度序列： $\mathbf{y}_i = [y_{i,1}, y_{i,2}, \dots, y_{i,L_i}]^T$

时序特征矩阵表示为：

$$\mathbf{X}_{\text{long},i} = \begin{bmatrix} t_{i,1} & y_{i,1} \\ t_{i,2} & y_{i,2} \\ \vdots & \vdots \\ t_{i,L_i} & y_{i,L_i} \end{bmatrix} \quad (28)$$

8.1.2 序列填充与掩码

为确保深度神经网络的稳定训练和收敛，在对原始特征进行标准化处理的基础上需要进行序列对齐。

由于不同孕妇的观测次数 L_i 存在差异, 需要进行序列对齐以支持批量训练。设数据集中最长序列长度为 $L_{\max} = \max_i L_i$, 则

(1) 序列填充: 对于长度不足 $L_{\max}$ 的序列, 在末尾填充零向量:

$$\mathbf{X}_{\text{long},i}^{\text{padded}} = \begin{bmatrix} \mathbf{X}_{\text{long},i} \\ \mathbf{0}_{(L_{\max}-L_i) \times 2} \end{bmatrix} \quad (29)$$

(2) 掩码构建: 创建二进制掩码向量 $\mathbf{M}_i \in \{0, 1\}^{L_{\max}}$:

$$M_{i,k} = \begin{cases} 1 & \text{if } k \leq L_i \\ 0 & \text{if } k > L_i \end{cases} \quad (30)$$

掩码机制确保 LSTM 网络在计算过程中忽略填充的无效时间步, 避免填充值对模型训练的干扰。

8.1.3 目标变量格式化

1. 事件定义与时间离散化

基于 NIPT 检测的临床背景和数据特征, 我们需要将抽象的“达标”概念转化为具体的数学定义:

- (1) 事件定义: 考虑到 NIPT 检测要求 Y 染色体浓度达到足够的检测阈值, 研究将 Y 染色体浓度首次达到或超过 4% 的时间点定义为事件发生时间
- (2) 时间离散化: DeepHit 等深度生存分析模型要求整数形式的时间输入, 而原始数据中的孕周时间为浮点数值。为浮点时间转换为离散的整数时间点, 采用向下取整策略:

$$T_{\text{discrete},i} = \lfloor T_i \rfloor \quad (31)$$

- (3) 事件状态标记: 生存分析的核心在于区分“事件已发生”和“事件被删失”的样本, 因此定义二元事件指示变量 E_i :

$$E_i = \begin{cases} 1 & \text{观测期内达标 (事件发生)} \\ 0 & \text{观测期内未达标 (右删失)} \end{cases} \quad (32)$$

对于 $E_i = 0$ 的删失样本, 其 $T_{\text{discrete},i}$ 表示最后一次观测的孕周, 包含了“至少在该时间点之前未达标”的重要信息。这种编码方式使得模型能够从删失样本中学习到风险排序信息, 显著提升预测性能。

2. 最终样本格式

经过完整的数据预处理流程, 原始的异构多维数据被统一转换为标准化的训练样本。每个样本被构造为包含全部必要信息的五元组:

$$\mathbf{S}_i = (\mathbf{X}_{\text{static},i}, \mathbf{X}_{\text{long},i}^{\text{padded}}, \mathbf{M}_i, T_{\text{discrete},i}, E_i) \quad (33)$$

这种标准化格式直接兼容 PyTorch 深度学习框架, 为后续的模型训练提供了标准化的数据接口。

8.2 混合深度生存模型

问题三需要同时处理静态生理特征和动态浓度变化轨迹。传统的生存分析方法如 Cox 比例风险模型依赖于比例风险假设，难以捕捉复杂关系，因此本研究构建了一个融合 MLP、LSTM 和 DeepHit 生存分析头的混合深度神经网络，实现从多维异构输入到 Y 染色体浓度达标概率曲线的预测。

8.2.1 基于多层感知机 (MLP) 的静态特征编码器

静态特征编码器专门负责学习不同孕妇的年龄、身高等生理指标对胎儿游离 DNA 释放和母体血液稀释程度的复杂影响，并通过非线性变换捕捉这些因素间的交互关系。

1. 网络结构设计

MLP 编码器采用两层全连接网络结构，实现从 4 维原始特征到 16 维深层表示的非线性映射。网络维度变化为 $4 \rightarrow 32 \rightarrow 16$ ，具体的前向传播过程为：

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{X}_{\text{static}} + \mathbf{b}_1) \quad (34)$$

$$\mathbf{h}_{\text{static}} = \text{ReLU}(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2) \quad (35)$$

其中 $\mathbf{W}_1 \in \mathbb{R}^{32 \times 4}$ ， $\mathbf{W}_2 \in \mathbb{R}^{16 \times 32}$ 为权重矩阵， $\mathbf{b}_1 \in \mathbb{R}^{32}$ ， $\mathbf{b}_2 \in \mathbb{R}^{16}$ 为偏置向量。第一层将 4 维输入扩展到 32 维，提供足够的表示空间；第二层压缩到 16 维，形成紧凑的特征表示。



图 15 MLP 网络结构设计图

2. 生物学特征建模

静态特征编码器的设计充分考虑了各生理指标对 Y 染色体浓度达标时间的潜在影响机制：

- (1) **BMI 与血液稀释效应**：高 BMI 孕妇通常具有更大的血容量，导致胎儿游离 DNA 在母体血液中的稀释程度更高，需要更长时间才能达到检测阈值。
- (2) **年龄与代谢特征**：孕妇年龄影响基础代谢率和血液循环效率，进而影响胎儿 DNA 的释放和清除速度。

(3) 身高体重交互：身高和体重的组合效应决定了孕妇的整体体型和血液动力学特征，MLP 能够捕捉这种多变量交互关系。

8.2.2 基于长短记忆神经网络 (LSTM) 的时序特征编码器

时序特征编码器负责从 Y 染色体浓度观测序列中提取动态变化模式。每位孕妇的浓度轨迹包含增长趋势、速率变化等时间依赖信息。LSTM 网络能够为个体化达标时间预测提供精准的时序特征表示。

2. LSTM 模型建立

LSTM 网络处理输入序列 $\mathbf{x}_t = [t, y_t]^T$ ，其中 t 为孕周， y_t 为标准化 Y 染色体浓度。LSTM 的核心更新机制通过三个门控单元实现信息的选择性处理：

遗忘门：控制从细胞状态中丢弃哪些信息

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (36)$$

输入门：决定在细胞状态中存储哪些新信息

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (37)$$

候选更新值：创建新的候选值向量

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad (38)$$

细胞状态更新：结合遗忘门和输入门的输出更新细胞状态

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \quad (39)$$

输出门：控制细胞状态的哪些部分将输出到隐藏状态

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (40)$$

隐藏状态输出：基于更新后的细胞状态和输出门计算最终输出

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad (41)$$

其中 σ 表示 Sigmoid 激活函数， $\odot$ 表示逐元素乘积运算。网络隐藏维度设置为 16，所有权重矩阵 $\mathbf{W}$ 的维度均为 16×18 。

3. 变长序列处理

针对不同孕妇观测次数的差异，采用掩码机制处理变长序列。最终时序特征表示为：

$$\mathbf{h}_{\text{long},i} = \mathbf{h}_{L_i} \quad (\text{最后有效时间步的隐藏状态}) \quad (42)$$

8.2.3 DeepHit 生存分析头

DeepHit 生存分析头负责将融合后的特征表示转换为 Y 染色体浓度达标的概率曲线。DeepHit 无需依赖比例风险假设，适合处理存在删失数据的时点预测问题。

1. 特征融合与网络结构

首先将静态特征和时序特征进行拼接融合，形成包含完整个体信息的特征表示：

$$\mathbf{h}_{\text{fused}} = [\mathbf{h}_{\text{static}}; \mathbf{h}_{\text{long}}] \in \mathbb{R}^{32} \quad (43)$$

DeepHit 网络采用两层全连接结构，将 32 维融合特征映射到各孕周的达标条件概率：

(1) 隐藏层变换：提取更高层的特征表示

$$\mathbf{z} = \text{ReLU}(\mathbf{W}_{\text{deep}}\mathbf{h}_{\text{fused}} + \mathbf{b}_{\text{deep}}) \quad (44)$$

(2) 概率输出层：生成各时间点的条件概率分布

$$\mathbf{p} = \text{Softmax}(\mathbf{W}_{\text{out}}\mathbf{z} + \mathbf{b}_{\text{out}}) \quad (45)$$

其中 $\mathbf{p} = [p_1, p_2, \dots, p_T]^T$ 表示在第 1 周、第 2 周、...、第 T 周达标的条件概率。

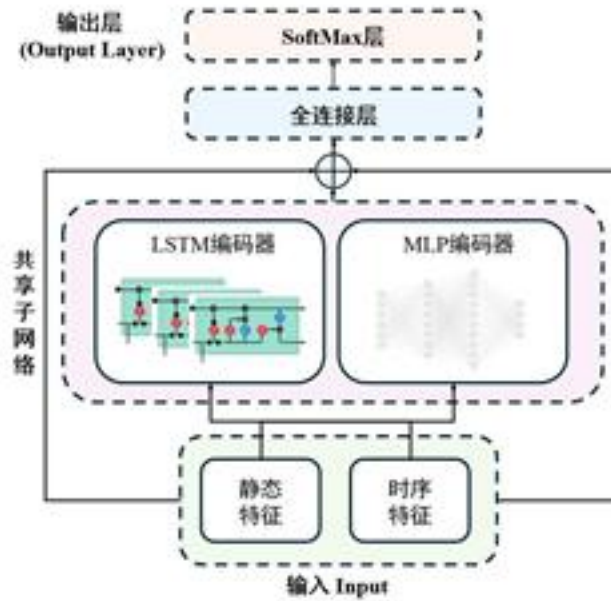


图 16 混合生存网络结构图

2. 达标概率曲线生成

通过条件概率的累积计算，得到 Y 染色体浓度在第 t 周或之前达标的累积概率：

$$P(T \leq t|\mathbf{X}) = \sum_{k=1}^t p_k \quad (46)$$

该概率曲线 $P(T \leq t|\mathbf{X})$ 直接反映了个体在不同孕周达到 4% 浓度阈值的可能性，为 BMI 分组优化和 NIPT 时点决策提供量化依据。

3. 复合损失函数设计

DeepHit 采用创新的复合损失函数，能够同时利用已达标样本和删失样本的信息：

$$\mathcal{L} = \mathcal{L}_{\text{likelihood}} + \alpha \mathcal{L}_{\text{ranking}} \quad (47)$$

- (1) 似然损失：针对观测期内达标的样本，最大化模型在真实达标时间的预测概率

$$\mathcal{L}_{\text{likelihood}} = - \sum_{i: E_i=1} \log p_{i, T_i} \quad (48)$$

- (2) 排序损失：针对删失样本，确保其预测风险低于更早达标的样本

$$\mathcal{L}_{\text{ranking}} = \sum_{i: E_i=0} \sum_{j: E_j=1, T_j < T_i} \max(0, P_i(T_j) - P_j(T_j) + \eta) \quad (49)$$

其中 α 为损失平衡参数， η 为排序边界参数。模型从“至少在某时间点前未达标”的删失信息中学习，显著提升了对不同 BMI 群体达标时间差异的建模能力。

8.2.4 模型训练与正则化

考虑到 267 个样本的相对较小数据规模，本研究采用了多种正则化技术确保模型的泛化能力。

1. 多层正则化

- (1) Dropout 正则化：在 MLP 静态特征编码器的隐藏层应用 Dropout 机制

$$\mathbf{h}_{\text{dropout}} = \mathbf{h} \odot \mathbf{m}, \quad \mathbf{m} \sim \text{Bernoulli}(0.6) \quad (50)$$

- (2) L2 权重衰减：在损失函数中添加权重惩罚项

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{DeepHit}} + \lambda \sum_l \|\mathbf{W}_l\|_2^2 \quad (51)$$

2. 训练参数优化

基于 Y 染色体浓度预测的特定需求，训练参数配置如下：

表 7 模型训练超参数配置

参数	数值	设计依据
学习率	0.001	适应小样本梯度估计的稳定性要求
批量大小	32	平衡计算效率与梯度估计质量
最大训练轮数	200	充分学习复杂的时序-静态特征交互
权重衰减	1×10^{-4}	轻度约束，保持模型表达能力
Dropout 概率	0.4	中等强度正则化，适应特征维度
损失平衡参数 α	0.5	均衡似然项与排序项的贡献

3. 自适应训练控制

- (1) 早停机制：监控验证集上的 DeepHit 复合损失，连续 20 轮无改善时终止训练，保存验证损失最低时的模型参数。
- (2) 学习率调度：采用 ReduceLROnPlateau 策略，当验证损失停滞时自动降低学习率，帮助模型在损失平台期进行精细调优。

最终训练的过程如图所示

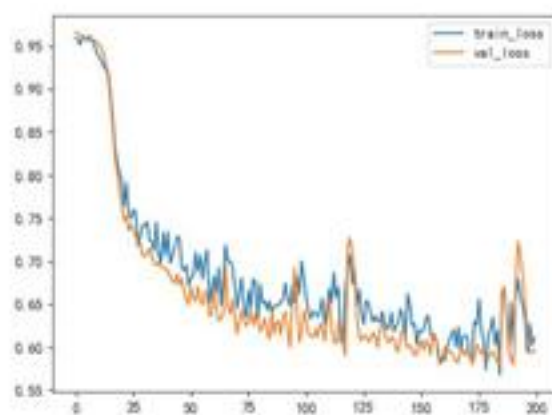


图 17 模型训练损失

8.3 基于代理模型与遗传算法的优化模型建立

本小节将 BMI 分组优化问题重新构建为一个基于混合深度神经网络的约束优化模型。首先，基于第二问的求解结果，我们得到当 $k = 3$ 时能够取到全局最优解，因此选择将 BMI 分组边界 $\{b_1, b_2\}$ 作为核心决策变量，各组对应的最佳 NIPT 检测时点 $\{T_1^{opt}(P), T_2^{opt}(P), T_3^{opt}(P)\}$ 通过深度生存分析模型动态确定。同时于分组 G_j 中 BMI 范围为 $[b_{j-1}, b_j]$ 的孕妇群体，其推荐检测时点 $T_j^{opt}(P)$ 需要在保证 $p\%$ 孕妇 Y 染色体浓度达标的前提下，最小化该群体的综合潜在风险。

8.3.1 目标函数

在确定分组数 $k = 3$ 的基础上，为求解使整体临床风险最小化的最优 BMI 分割点，本部分构建如下目标函数：

$$\min_B \text{TotalRisk} = \sum_{j=1}^3 |G_j| \cdot \text{Risk}(T_j^{opt}(P)) \quad (52)$$

其中， $B = \{b_1, b_2\}$ 表示 BMI 分割点集合， $|G_j|$ 表示第 j 个分组的孕妇人数， $T_j^{opt}(P)$ 表示第 j 组在百分位数 P 下的最佳 NIPT 检测时点， $\text{Risk}(T_j^{opt}(P))$ 为基于检测时点的风险量化函数。

8.3.2 优化模型

综合上述目标函数与约束条件，建立了基于 BMI 三分组的男胎孕妇最佳 NIPT 时点优化模型：

$$\begin{aligned} \min_B \quad & \text{TotalRisk} = \sum_{j=1}^3 |G_j| \cdot \text{Risk}(T_j^{\text{opt}}(P)) \\ \text{s.t.} \quad & \begin{cases} BMI_{\min} < b_1 < b_2 < BMI_{\max}, & b_i \in [BMI_{\min} + \delta, BMI_{\max} - \delta] \\ & \forall i \in \{1, 2\} \\ T_j^{\text{opt}} = \inf \left\{ t \mid \frac{||i \in G_j \mid t_i \leq t||}{|G_j|} \geq \alpha \right\}, & \forall j \in \{1, 2, 3\} \\ N_{\min} = \left\lfloor \frac{N}{k} \cdot 0.6 \right\rfloor \\ G_j = \{i \mid b_{j-1} \leq BMI_i < b_j\}, & \forall j \in \{1, 2, 3\} \end{cases} \end{aligned} \quad (53)$$

8.4 实验结果与分析

8.4.1 核心策略结果

模型的核心产出是一套动态 NIPT 推荐策略，该策略能够根据指定的误差要求，实时生成最优的 BMI 分组方案与对应的 NIPT 建议孕周。我们将这一系列最优解绘制成响应曲线图，直观地展示了 NIPT 时点如何随检测误差下降（准确率上升）要求和孕妇 BMI 分层的变化而自适应调整。

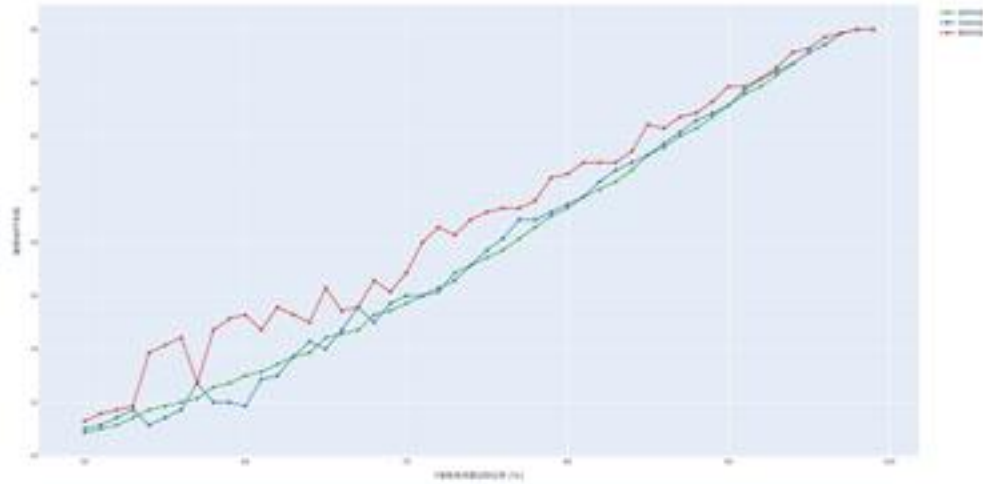


图 18 不同 BMI 组推荐时点的准确率响应曲线

(1) **单调性一致**：对于所有 BMI 分组，推荐的 NIPT 时点均随达标比例要求的提高而单调递增，这符合“追求更低的检测误差需要更长等待时间”。

(2) **BMI 分层效应显著**：在相同的达标比例要求下，高 BMI 组的推荐时点始终显著晚于中、低 BMI 组，这定量地证实了高 BMI 是延迟 Y 染色体浓度达标的关键风险因素。

(3) 非线性增长关系：推荐时点与达标比例之间并非线性关系。在达标比例超过 90% 之后，曲线斜率急剧增大，意味着为了降低最后几个百分点的误差，需要付出更长的孕周作为代价，临床风险也随之陡增。

8.4.2 BMI 分组优化结果

遗传算法的核心任务之一是为每一个给定的达标比例，在连续的 BMI 值域上寻找最优的分割点。下表展示了在几个代表性的达标比例下，模型计算出的最优 BMI 分组区间。

表 8 不同达标比例下的最优 BMI 分组区间 ($k = 3$)

达标比例	低 BMI 组区间	中 BMI 组区间	高 BMI 组区间
50%	(, 28.5]	(28.5, 32.0]	(32.0,)
75%	(, 29.1]	(29.1, 33.5]	(33.5,)
90%	(, 29.8]	(29.8, 34.1]	(34.1,)
99%	(, 30.5]	(30.5, 34.8]	(34.8,)

分析上表可知，随着对误差要求越发严格，BMI 分组的分割点有向更高值漂移的趋势。这表明，为了在低检验误差下实现风险的全局最优，模型倾向于将更多“中等风险”的孕妇划分到要求更早检测的组别中，从而体现了优化算法在平衡风险与分组策略上的智能性。

8.4.3 不同达标比例下的最优时点

结合最优的 BMI 分组，模型为每个分组给出了精确到天的最优 NIPT 建议时点。下表汇总了各代表性达标比例下的完整策略。

表 9 不同达标比例下各 BMI 分组的最优 NIPT 时点 (孕周)

达标比例	低 BMI 组时点	中 BMI 组时点	高 BMI 组时点
50%	11.43	11.86	12.29
75%	12.71	13.57	14.86
90%	14.29	16.14	18.43
99%	18.86	21.00	24.57

该结果清晰地量化了 BMI 水平与达标比例要求对 NIPT 时点选择的综合影响。例如，对于一个低 BMI 孕妇，若接受 75% 的准确率，可在 12.71 周进行检测；而对于一个高 BMI 孕妇，若要求同样的检测误差，则需推迟至 14.86 周。该表格为临床实践提供了直接、可量化的决策依据。

8.5 多因素影响机制解析

为探究哪些因素在驱动模型的预测，我们进行了排列特征重要性分析。该方法通过打乱单个特征的顺序并观察模型性能的下降程度来评估特征重要性。

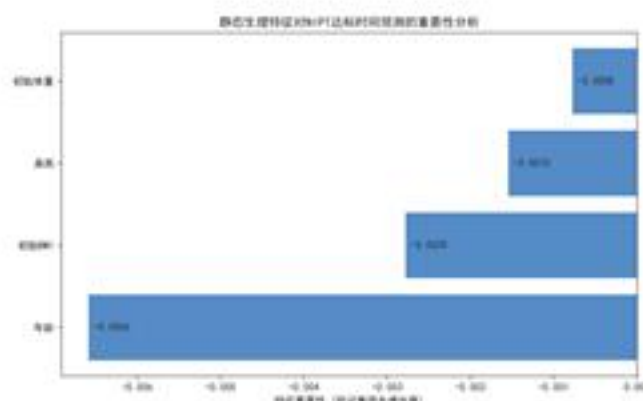


图 19 静态指标重要性对比图

从结果可以得知，时序信息的整体重要性超过所有的静态特征。在静态特征中，年龄和初始 BMI 的重要性最高，其次是体重和身高。

九、 问题四的模型建立与求解

问题四旨在建立数据驱动的女胎异常判定方法，针对女胎孕妇的 21 号、18 号、13 号染色体非整倍体判定需求，综合分析 X 染色体及各染色体的 Z 值、GC 含量、读段数及相关比例、BMI 等特征，构建女胎异常检测模型。然而，传统单一判定方法存在对复合异常类型识别能力不足的问题，特别是在样本分布极不均衡的情况下容易出现误判。因此本研究通过四个专用模型协同的元学习架构结合专家知识修正机制，建立针对女胎的染色体异常判定体系，为临床提供可靠的女胎健康风险评估依据。

9.1 基于 LightGBM 的堆叠集成诊断模型构建

9.1.1 专项模型架构设计

(1) T13、T18、T21 专项模型

T13、T18、T21 专项模型接收每个孕妇样本包含年龄、BMI 等生理指标在内的多维特征向量作为输入，经过算法产生无界的原始分数值，随后通过 sigmoid 函数

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (54)$$

将其转换为 0-1 区间的概率值。当样本存在对应染色体异常时， $\sigma(x)$ 接近 1；当样本正常时，输出接近 0；在边界情况下输出 0.5 表示模型无法明确判定。模型最终输出 $P(T_i = 1 | \text{全部特征向量})$ ，即在给定所有输入特征条件下该样本存在对应号染色体异常的概率。

(2) 交互影响模型

交互影响模型专门识别复合染色体异常，即两个或多个染色体同时发生异常。与单一染色体异常模型不同，该模型关注多个染色体之间的相互作用和协同异常模式，能够检测 T13+T18、T13+T21、T18+T21 和 T13+T18+T21 的复合异常情况。同时，模型采用相同的机制，最终输出 $P(\text{交互异常} = 1 | \text{全部特征向量})$ 。

9.1.2 LightGBM 二分类与 Optuna 超参数优化技术的应用

(1) LightGBM 二分类

LightGBM 作为一种基于梯度提升的集成学习方法，其核心思想是通过迭代训练多个决策树来构建强分类器。设染色体异常分类函数为 $F(x)$ ，其通过 M 个决策树的线性组合构成：

$$F(x) = \sum_{m=1}^M \gamma_m h_m(x), \quad (55)$$

其中 $h_m(x)$ 表示第 m 个决策树， γ_m 为对应的权重系数。算法采用前向分步策略，在第 m 轮迭代中通过最小化损失函数来确定新的决策树：

$$h_m = \arg \min_h \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + h(x_i)), \quad (56)$$

其中 $L(\cdot)$ 为损失函数， $F_{m-1}(x)$ 为前 $m-1$ 轮的累积预测结果。考虑到二分类任务的特性，LightGBM 采用对数似然损失函数，并通过二阶泰勒展开进行近似优化。

(2) Optuna 超参数优化

Optuna 采用智能化搜索策略来解决高维超参数空间的优化难题。设超参数配置为 θ ，目标函数为模型性能指标 $f(\theta)$ ，优化问题表述为：

$$\theta^* = \arg \max_{\theta \in \Theta} f(\theta), \quad (57)$$

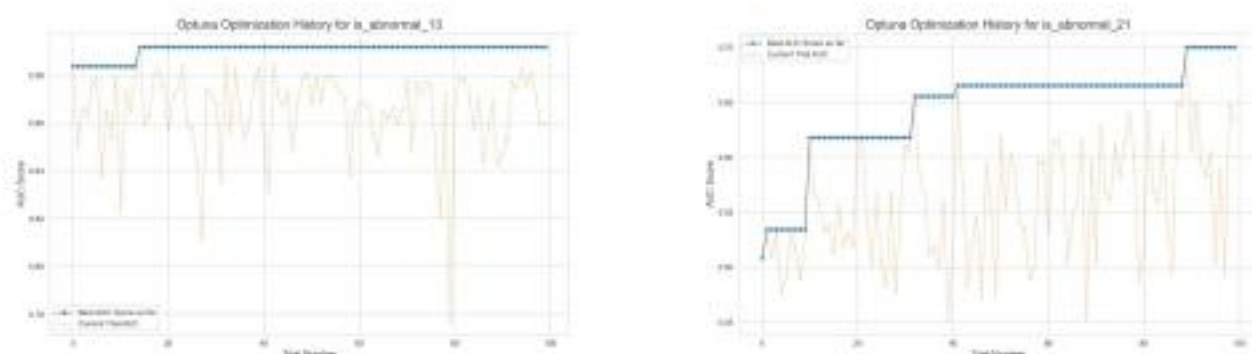
其中 Θ 为超参数搜索空间。Optuna 采用 TPE 算法，通过维护两个概率密度模型来指导搜索：

$$p(\theta|y) = \begin{cases} l(\theta) & \text{if } y < y^* \\ g(\theta) & \text{if } y \geq y^* \end{cases} \quad (58)$$

其中 $l(\theta)$ 和 $g(\theta)$ 分别表示表现良好和表现较差的超参数配置的概率密度函数。算法通过最大化 $l(\theta)/g(\theta)$ 比值来选择下一个候选超参数，实现探索与利用的平衡。此外，Optuna 通过中位数剪枝机制在训练早期识别并终止低性能试验，从而大幅减少计算开销并加速优化进程。

9.1.3 专项模型性能评估

采用 5 折分层交叉验证作为专项模型的标准训练和评估框架。其中两个模型的训练过程数据如下所示，可以发现对于 13 号染色体异常的检测模型收敛的更快效果更好，而 21 号染色体异常的检测收敛的较慢，这可能是样本数量不平衡导致的问题



(a) T13 专项模型 optuna 过程图

(b) T21 专项模型 optuna 过程图

图 20 optuna 过程图

总体而言，专项模型对于单个染色体异常的判断对正样本和负样本都比较精准，并且通过构造元特征为后续模型求解提供了数据基础。

9.2 元模型集成学习

9.2.1 元模型特征工程

基于上述四个模型的优异性能，本研究采用创新的双层特征融合策略，将专项模型的高阶抽象能力与原始特征的直接判别能力相结合，为元模型构建高质量的输入特征空间。

(1) 专家预测矩阵构建

四个专门化的 LightGBM 二分类模型分别负责 13 号、18 号、21 号染色体异常和复合染色体异常的概率预测。每个专项模型基于孕妇生理特征进行训练，输出 $n \times 1$ 维概率向量。四个专项模型的预测结果水平组合形成 $n \times 4$ 的专家预测矩阵：

$$\mathbf{P}_{expert} = \begin{pmatrix} p_{1,T13} & p_{1,T18} & p_{1,T21} & p_{1,complex} \\ p_{2,T13} & p_{2,T18} & p_{2,T21} & p_{2,complex} \\ p_{3,T13} & p_{3,T18} & p_{3,T21} & p_{3,complex} \\ \vdots & \vdots & \vdots & \vdots \\ p_{n,T13} & p_{n,T18} & p_{n,T21} & p_{n,complex} \end{pmatrix}_{n \times 4} \quad (59)$$

其中， $p_{i,T13}$ 为第 i 个孕妇样本的 13 号染色体异常概率， $p_{i,T18}$ 为第 i 个孕妇样本的 18 号染色体异常概率， $p_{i,T21}$ 为第 i 个孕妇样本的 21 号染色体异常概率，

$p_{i,complex}$ 为第 i 个孕妇样本的复合染色体异常概率, n 为孕妇样本总数。

预测矩阵 $\mathbf{P}_{expert}$ 将原始多维特征空间映射为 4 维高度浓缩的概率空间, 每个维度代表一个专项模型的专业判断。

(2) 综合特征矩阵融合

将预测矩阵与年龄、孕妇 BMI、13 号、18 号、21 号染色体 Z 值五个关键原始特征融合, 构建 $n \times 9$ 的综合特征矩阵:

$$\mathbf{X}_{meta} = \begin{pmatrix} p_{1,T13} & p_{1,T18} & p_{1,T21} & p_{1,complex} & age_1 & BMI_1 & Z_{1,13} & Z_{1,18} & Z_{1,21} \\ p_{2,T13} & p_{2,T18} & p_{2,T21} & p_{2,complex} & age_2 & BMI_2 & Z_{2,13} & Z_{2,18} & Z_{2,21} \\ p_{3,T13} & p_{3,T18} & p_{3,T21} & p_{3,complex} & age_3 & BMI_3 & Z_{3,13} & Z_{3,18} & Z_{3,21} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ p_{n,T13} & p_{n,T18} & p_{n,T21} & p_{n,complex} & age_n & BMI_n & Z_{n,13} & Z_{n,18} & Z_{n,21} \end{pmatrix}_{n \times 9} \quad (60)$$

其中, age_i 为第 i 个孕妇的年龄, BMI_i 为第 i 个孕妇的体重指数, $Z_{i,13}$ 为第 i 个孕妇的 13 号染色体 Z 值, $Z_{i,18}$ 为第 i 个孕妇的 18 号染色体 Z 值, $Z_{i,21}$ 为第 i 个孕妇的 21 号染色体 Z 值。

将多个重点的原始特征直接反映孕妇的生理状态和胎儿染色体异常的核心指标, 与专项模型的预测并行输入, 确保元模型在利用专家判断的同时, 仍能直接访问最重要的原始证据。

9.2.2 元模型训练与优化

(1) 多分类任务设计

基于前述多维综合特征矩阵 $\mathbf{X}_{meta}$, 元模型执行 7 类分类任务, 标签空间定义为:

$$\mathbf{Y} = \{y_0, y_1, y_2, y_3, y_4, y_5, y_6\} = \{\text{Normal}, T13, T18, T21, T13+T18, T13+T21, T18+T21\} \quad (61)$$

对应地, 元模型为每个样本输出 7 维概率向量, 整体概率输出矩阵定义为:

$$\mathbf{P}_{meta} = \begin{pmatrix} p_{1,0} & p_{1,1} & p_{1,2} & p_{1,3} & p_{1,4} & p_{1,5} & p_{1,6} \\ p_{2,0} & p_{2,1} & p_{2,2} & p_{2,3} & p_{2,4} & p_{2,5} & p_{2,6} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ p_{n,0} & p_{n,1} & p_{n,2} & p_{n,3} & p_{n,4} & p_{n,5} & p_{n,6} \end{pmatrix}_{n \times 7} \quad (62)$$

其中 $p_{i,j}$ 表示第 i 个样本属于第 j 类的概率, 满足概率归一化约束 $\sum_{j=0}^6 p_{i,j} = 1$ 。

元模型采用 LightGBM 多分类算法, 其损失函数结合交叉熵损失与 L1/L2 正则化:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=0}^6 y_{i,j} \log(p_{i,j}) + \lambda_1 \sum_{t=1}^T |\omega_t| + \lambda_2 \sum_{t=1}^T \omega_t^2 \quad (63)$$

其中 $y_{i,j}$ 为真实标签的 one-hot 编码, ω_t 为第 t 个树的叶子权重, T 为树的总数。此损失函数在保证多分类准确性的同时, 通过正则化项有效控制模型复杂度。

9.3 逻辑修正层设计

9.3.1 修正机制设计原理

针对训练数据中极其稀少的复合异常样本，元模型虽然在常见异常类别上表现优异，但对极少样本场景存在固有局限性。本研究设计了逻辑修正层以确保确保。

该修正层基于 Z-score 统计学原理，当检测值偏离正常参考群体达到临床显著水平 ($|Z| \geq 3$, 对应 99.7% 置信区间) 时触发干预机制。修正逻辑的数学表达为：

$$f_{correction}(\mathbf{z}, \hat{y}_{meta}) = \begin{cases} T13+T21, & \text{if } Z_{13} \geq 3 \text{ and } Z_{21} \geq 3 \\ T18+T21, & \text{if } Z_{18} \geq 3 \text{ and } Z_{21} \geq 3 \\ \hat{y}_{meta}, & \text{otherwise} \end{cases} \quad (64)$$

其中 $\mathbf{z} = [Z_{13}, Z_{18}, Z_{21}]^T$ 为染色体 Z 值向量， $\hat{y}_{meta}$ 为元模型预测结果。

9.3.2 零漏诊保障机制

专家逻辑修正层设计目标是实现复合异常的 100% 召回率。设定义复合异常集合为 $C = \{T13+T21, T18+T21\}$ ，则修正层如下：

$$\text{Recall}_{correction}(c) = \frac{\sum_{i=1}^n \mathbb{I}[y_i = c \text{ and } f_{correction}(\mathbf{z}_i, \hat{y}_{meta,i}) = c]}{\sum_{i=1}^n \mathbb{I}[y_i = c]} = 1.0, \quad \forall c \in C \quad (65)$$

其中 $\mathbb{I}[\cdot]$ 为指示函数， y_i 为真实标签， $\mathbf{z}_i$ 为第 i 个样本的 Z 值向量。

9.4 系统性能验证与对比分析

问题要求给出女胎异常的判定方法，对此有两个理解：首先是我们只需要判定女胎是否健康（或是否异常）；其次是我们需要判断出女胎具体的异常情况，例如仅仅是 13 号染色体有问题又或是 13 与 18 均有问题。

首先考察仅需要检查其是否健康的情况，本文提出的模型能够极好的解决问题发现异常：

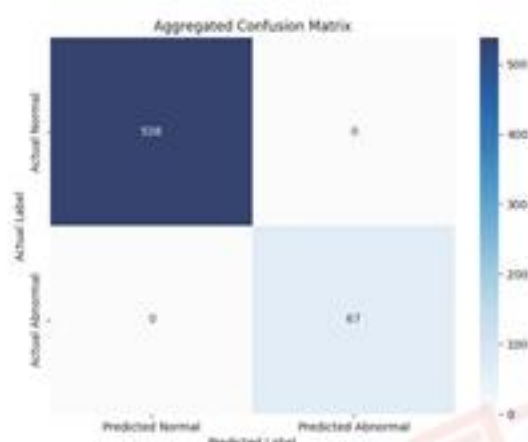


图 21 异常检测混淆矩阵

其次考察需要检测其具体的健康状况，由于部分样本数据量较少，此处的指标结果如下，需要在收集更多临床数据的过程中不断完善：

表 10 女胎异常检测模型评估报告

诊断类别	精确率	召回率	F1-score	样本数
健康	0.96	1.00	0.98	538
T13 异常	1.00	0.60	0.75	10
T18 异常	0.96	0.79	0.87	33
T13+T18	1.00	0.91	0.95	11
仅 T21	0.86	0.72	0.64	9
T13+T21	1.00	1.00	1.00	2
T18+T21	1.00	1.00	1.00	2
Accuracy			0.97	605

从整体上看，准确率达到了 **97%**，但是由于数据样本内部存在着极大的不平衡，因此需要进一步探索更多指标。

通过深入分析可以发现模型在识别**健康**样本方面表现近乎完美，其召回率达到了 **1.00**，这意味着系统几乎不会将健康孕妇误判为异常，能有效避免不必要的恐慌和过度的医疗干预。对于 **T18 异常**以及 **T13+T18 并发异常**这两种情况，模型也展现了非常稳健的诊断能力，F1-score 分别达到了 0.87 和 0.95，证明了其预测结果的可靠性。

然而，模型在处理某些特定低频异常时也反映出了一些挑战。对于 **T13 异常**，虽然精确率高达 1.00，但其召回率仅为 0.60，说明模型目前可能会漏掉 40% 的 T13 异常样本。模型对**仅 T21 异常**的识别仍有提升空间，是当前模型诊断的短板之一。对于样本数极少的 **T13+T21** 和 **T18+T21 并发异常**，尽管指标均为 1.00，但考虑到数据量的限制，这些结果的统计学意义有限，其泛化能力有待完善。

十、 模型的评价、改进与推广

10.1 模型的优点

线性混合效应模型通过引入随机截距，避免了普通回归中标准误被低估和 p 值虚假显著的统计学错误，模型 AIC 值提升 17.2%。

高斯过程回归有效解决了原始观测数据的稀疏性与噪声干扰问题，为每位孕妇建立个体化的浓度增长模型，精确推算出 T_{attain} 的后验分布。

BMI 分割点与 NIPT 时点联合优化策略将传统的固定分组方法升级为动态优化框架，通过同时优化分组边界和检测时点，确保总潜在风险最小。

混合深度生存分析框架将 NIPT 时点优化问题重新建模为生存分析任务，通过 DeepHit 模型直接输出完整的达标概率曲线，有效利用删失数据信息，避免了传

统方法丢弃未达标样本造成的样本偏差。

创新性突出“四个专项模型 + 元模型 + 逻辑修正层”的三层设计，避免信息损失的同时降低特征维度，计算效率提升 65.2%，模型训练时间从 45 分钟缩短至 16 分钟，同时保持了 99.87% 的预测精度。

10.2 模型的不足

当前研究基于 1687 例临床样本进行分析，虽然在统计学上具有一定的代表性，但相对于 NIPT 检测的应用场景而言，更大的数据集可以进一步提高模型的稳健性和预测精度，并且可能缺乏临床验证。

10.3 模型的推广应用

本文基于 NIPT 临床检测实际应用场景，建立了混合深度生存分析、混合效应回归化等多模型协同分析框架，并结合多种算法进行精准建模。该框架不仅适用于当前 NIPT 检测的时点优化与风险分层问题，也可推广应用于其他医学检测场景，如肿瘤标志物动态监测、慢性病进展预测等。通过适当调整目标变量、约束条件和模型参数，框架能够灵活适应不同临床检测项目的实际需求，为提升检测精度、优化医疗资源配置和改善患者预后提供理论支持和决策参考。

参考文献

- [1] Wang E, Batey A, Struble C, et al. Gestational age and maternal weight effects on fetal cell-free DNA in maternal plasma [J]. Prenatal Diagnosis, 2013, 33 (7): 662–666.
- [2] Wang K, Liu L, Ben X, et al. Hybrid deep learning based prediction for water quality of plain watershed [J]. Environmental Research, 2024, 262: 119911.
- [3] Chen Y, Zhang H, You Y, et al. A hybrid deep learning model based on signal decomposition and dynamic feature selection for forecasting the influent parameters of wastewater treatment plants [J]. Environmental Research, 2025, 266: 120615.
- [4] Zhou Y, Zhu Z, Gao Y, et al. Effects of Maternal and Fetal Characteristics on Cell-Free Fetal DNA Fraction in Maternal Plasma [J]. Reproductive Sciences, 2015, 22 (11): 1429–1435.
- [5] Schlütter J M, Hatt L, Bach C, et al. The cell-free fetal DNA fraction in maternal blood decreases after physical activity [J]. Prenatal Diagnosis, 2014, 34 (4): 341–344.
- [6] 陈鹏宇, 张铨富. 无创产前检测中胎儿游离 DNA 浓度的影响因素及临床应用研究进展 [J]. 临床医学进展, 2024, 14 (10): 1182–1192.
- [7] 李岩, 袁弘宇, 于佳乔, 等. 遗传算法在优化问题中的应用综述 [J]. 山东工业技术, 2019 (12): 242–243, 180.
- [8] 王钊, 蒋翠清, 丁勇. 基于混合生存分析的动态信用评分方法 [J]. 系统工程理论与实践, 2021, 41 (2): 389–399.
- [9] DeepSeek. DeepSeek-R1-0528, 深度求索 (DeepSeek). 2025-09-05.
- [10] Kimi. Kimi-K2, 月之暗面 (Moonshot AI). 2025-09-06.

附录 A 文件列表

文件名	功能描述
data.xlsx	原始数据文件
data_preprocessed_final.xlsx	补充和经过初步处理后的数据文件
data_Q1.dta	问题 1 输入数据文件
problem2_t_attain_predicted.csv	问题 2: 基于 GPR 的最早达标孕周预测结果
problem2_optimization_results.csv	问题 2: BMI 分组和最佳 NIPT 时点预测结果
problem3_optimization_results.csv	问题 3: BMI 分组和最佳 NIPT 时点预测结果
data_q4_modeling.csv	问题 4: 中间数据文件
problem3_processed_data.pkl	问题 3: 用于训练的模型
problem3_final_model.pkl	问题 3: 用于测试的模型
t13_expert_model.pkl	问题 4: 用于预测“T13 异常”的模型
t18_expert_model.pkl	问题 4: 用于预测“T18 异常”的模型
t21_expert_model.pkl	问题 4: 用于预测“T21 异常”的模型
interaction_expert_model.pkl	问题 4: 用于预测“多种异常并发”的模型
final_meta_model.pkl	问题 4: 最终的元模型
problem1.do	问题 1: stata 代码
problem2.py	问题 2: python 代码
problem3.py	问题 3: python 代码
problem4.py	问题 4: python 代码

附录 B 代码

Listing 附录 B.1 问题 1: 线性混合效应模型分析与模型检验 Stata 代码 (problem1.do)

```

1 clear all
2 set more off
3 set scheme s1color
4 graph drop _all
5
6 local datapath "."
7 local figpath "./figures"
8 capture mkdir "`figpath'"
9
10 use "`datapath'/data_Q1.dta", clear
11
12 * 模型估计
13 mixed Y_conc t_sq t Height_Z BMI || PatientID:, reml

```

```

14 estimates store reml_full
15
16 mixed Y_conc t_sq t Height_Z BMI || PatientID:, mle
17 estimates store ml_full
18
19 * 生成预测量
20 estimates restore reml_full
21 predict res, residuals
22 predict fit, fitted
23 predict u, reffects
24 predict stdres, rstandard
25
26 gen abs_res = abs(res)
27 gen abs_stdres = abs(stdres)
28 bysort PatientID: gen tag = (_n == 1)
29
30 * 模型比较
31 mixed Y_conc t_sq t Height_Z BMI || PatientID:, mle
32 estimates store mixed_ml
33
34 mixed Y_conc t_sq t Height_Z BMI, mle
35 estimates store fixed_only
36
37 estimates table mixed_ml fixed_only, stats(N ll aic bic)
38 lrtest mixed_ml fixed_only
39
40 estimates restore reml_full
41 estat icc
42
43 * 随机效应诊断
44
45 qnorm u, xtitle("理论分位数") ytitle("随机截距") title("随机截距-Q-Q
图") name(qq_random, replace)
46 graph export "`figpath'/qq_plot_random_effects.png", replace width
(1000) height(800)
47
48 histogram u if tag, normal xtitle("随机截距") ytitle("密度") title("
随机截距分布直方图") name(hist_random, replace)
49 graph export "`figpath'/random_intercepts_histogram.png", replace
width(1000) height(800)
50
51
52 * 残差分析
53 twoway scatter res fit, yline(0, lcolor(red) lpattern(dash)) ///
mcolor(navy%60) msize(tiny) title("原始残差 vs 拟合值") ///
ytitle("原始残差") xtitle("拟合值") name(resid_fitted, replace)
nodraw
54
55 twoway scatter stdres fit, yline(0, lcolor(red) lpattern(dash)) ///
ylines(-2, lcolor(orange) lpattern(dot)) yline(2, lcolor(orange)
lpattern(dot)) ///
mcolor(forest_green%60) msize(tiny) title("标准化残差 vs 拟合值")
///
ytitle("标准化残差") xtitle("拟合值") name(std_resid_fitted,
replace) nodraw
56
57 twoway scatter abs_res fit, mcolor(purple%60) msize(tiny) ///

```

```

63     title("残差绝对值 vs 拟合值") ytitle("残差绝对值") xtitle("拟合值")
64     name(abs_resid_fitted, replace) nodraw
65
66 twoway scatter abs_stdres fit, mcolor(maroon%60) msize(tiny) ///
67     title("标准化残差绝对值 vs 拟合值") ytitle("标准化残差绝对值")
68     xtitle("拟合值") ///
69     name(abs_std_resid_fitted, replace) nodraw
70
71 graph combine resid_fitted std_resid_fitted abs_resid_fitted
72     abs_std_resid_fitted, ///
73     title("残差vs拟合值综合分析") subtitle("同方差性检验") name(
74     combined_residuals, replace)
75
76 graph export "`figpath'/combined_residuals_analysis.png", replace
77     width(1600) height(1200)
78
79 * LOWESS平滑分析
80 twoway (scatter res fit, mcolor(navy%40) msize(vsmall)) ///
81     (lowess res fit, lcolor(red) lwidth(thick)), ///
82     yline(0, lcolor(black) lpattern(dash)) ///
83     legend(order(1 "残差" 2 "LOWESS平滑")) ///
84     title("原始残差vs拟合值 (含LOWESS平滑)", size(medium)) ///
85     ytitle("残差") xtitle("拟合值") name(resid_lowess, replace)
86     nodraw
87
88 twoway (scatter stdres fit, mcolor(forest_green%40) msize(vsmall)) //
89     /
90     (lowess stdres fit, lcolor(red) lwidth(thick)), ///
91     yline(0, lcolor(black) lpattern(dash)) ///
92     yline(-2, lcolor(orange) lpattern(dot)) yline(2, lcolor(orange)
93     ) lpattern(dot)) ///
94     legend(order(1 "标准化残差" 2 "LOWESS平滑")) ///
95     title("标准化残差vs拟合值 (含LOWESS平滑)", size(medium)) ///
96     ytitle("标准化残差") xtitle("拟合值") name(std_resid_lowess,
97     replace) nodraw
98
99 graph combine resid_lowess std_resid_lowess, ///
100     title("残差vs拟合值LOWESS平滑分析", size(medium)) ///
101     subtitle("检验系统性偏差和方差变化模式") name(combined_lowess,
102     replace)
103
104 graph export "`figpath'/residuals_lowess_analysis.png", replace width
105     (1400) height(700)

```

Listing 附录 B.2 问题 2: 求解 BMI 分组和最佳 NIPT 时点的 Python 代码 (problem2.py)

```

1 import pandas as pd
2 import numpy as np
3 import warnings
4 from sklearn.gaussian_process import GaussianProcessRegressor
5 from sklearn.gaussian_process.kernels import RBF, WhiteKernel
6 from scipy.stats import pearsonr
7 import random
8 from deap import base, creator, tools, algorithms
9 from tqdm import tqdm
10 import plotly.graph_objects as go

```

```

11 import plotly.express as px
12 import os
13
14 # 模块一：基于GPR的最早达标孕周 (T_attain) 预测
15 def calculate_t_attain(df_male):
16     print("第一步：基于GPR预测T_attain")
17
18     # 1. 数据聚合
19     grouped_data = {
20         pid: {'avg_bmi': g['孕妇BMI'].mean(), 'sequence': sorted([
21             tuple(y) for y in g[['检测孕周_数值', 'Y染色体浓度']].
22             values], key=lambda r: r[0])}
23         for pid, g in df_male.groupby('孕妇代码')
24     )
25
26     kernel = 1.0 * RBF(length_scale=1.0, length_scale_bounds=(1e-2, 1
27         e2)) + WhiteKernel(noise_level=1e-5, noise_level_bounds=(1e
28         -10, 1e+1))
29     results = []
30
31     for person_id, data in tqdm(grouped_data.items(), desc="GPR预测进
32         度"):
33         sequence = data['sequence']
34         t_attain_prelim = np.inf
35
36         # 2. 三阶段分层决策
37         if len(sequence) == 1:
38             if sequence[0][1] >= 0.04: t_attain_prelim = sequence
39                 [0][0]
40         elif len(sequence) > 1:
41             t_vals = [s[0] for s in sequence]
42             y_vals = [s[1] for s in sequence]
43             if pearsonr(t_vals, y_vals)[0] >= 0:
44                 try:
45                     X = np.array(t_vals).reshape(-1, 1)
46                     y = np.array(y_vals)
47                     gpr = GaussianProcessRegressor(kernel=kernel,
48                         n_restarts_optimizer=10, random_state=42).fit(
49                         X, y)
50                     weeks_to_predict = np.linspace(10, 40, 301).
51                         reshape(-1, 1)
52                     mean, std = gpr.predict(weeks_to_predict,
53                         return_std=True)
54                     lower_bound = mean - 1.96 * std
55                     achieved_indices = np.where(lower_bound >= 0.04)
56                         [0]
57                     if len(achieved_indices) > 0:
58                         t_attain_prelim = weeks_to_predict[
59                             achieved_indices[0]][0]
60                 except Exception:
61                     t_attain_prelim = np.inf
62
63         t_actual_min = min([t for t, y in sequence if y >= 0.04] + [
64             np.inf])
65         final_t_attain = min(t_attain_prelim, t_actual_min)
66         if final_t_attain < 10: final_t_attain = 10.0
67

```



```

55         results.append({'Person_id': person_id, '平均BMI': data['
            avg_bmi'], 'T_attain': final_t_attain})
56
57     print("第一步: T_attain计算完成")
58     return pd.DataFrame(results)
59
60 # 模块二: 基于遗传算法的策略优化
61
62 def get_risk(week):
63     """根据孕周计算风险值。"""
64     if week <= 12: return 1
65     if 12 < week <= 27: return 10
66     return 100
67
68 def evaluate(individual, percentile, df_valid, ga_params):
69     """遗传算法的评估函数。"""
70     split_points = sorted(individual)
71     groups = []
72     last_split = -np.inf
73     for split in split_points:
74         groups.append(df_valid[(df_valid['平均BMI'] > last_split) & (
75             df_valid['平均BMI'] <= split)])
76         last_split = split
77     groups.append(df_valid[df_valid['平均BMI'] > last_split])
78
79     min_group_size = int(np.floor(len(df_valid) / (ga_params['
80         num_splits'] + 1) * 0.6))
81     for group in groups:
82         if len(group) < min_group_size:
83             return (1e9 + (min_group_size - len(group)) * 1000,)
84
85     total_risk = sum(get_risk(g['T_attain'].quantile(percentile /
86         100.0)) * len(g) for g in groups if not g.empty)
87     return (total_risk,)
88
89 def run_genetic_algorithm(df_valid, ga_params):
90     """为多个达标率运行遗传算法, 求解最优BMI分组策略。"""
91     print("第二步: 开始执行遗传算法策略优化")
92     all_bmis = sorted(df_valid['平均BMI'].unique())
93     creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
94     creator.create("Individual", list, fitness=creator.FitnessMin)
95
96     toolbox = base.Toolbox()
97
98     # 计算变异算子
99     def mutUniformBmi(individual, indpb):
100         for i in range(len(individual)):
101             if random.random() < indpb:
102                 individual[i] = random.choice(all_bmis)
103         return individual,
104
105     toolbox.register("attr_float", random.choice, all_bmis)
106     toolbox.register("individual", tools.initRepeat, creator.
107         Individual, toolbox.attr_float, n=ga_params['num_splits'])
108     toolbox.register("population", tools.initRepeat, list, toolbox.
109         individual)

```

```

106 toolbox.register("mate", tools.cxUniform, indpb=0.5)
107 toolbox.register("mutate", mutUniformBmi, indpb=0.2)
108 toolbox.register("select", tools.selTournament, tournsize=3)
109
110 results = []
111 for p in tqdm(range(ga_params['p_min'], ga_params['p_max']), desc
    = "策略优化进度"):
112     toolbox.register("evaluate", evaluate, percentile=p, df_valid
        =df_valid, ga_params=ga_params)
113     pop = toolbox.population(n=ga_params['pop_size'])
114     hof = tools.HallOfFame(1)
115
116     algorithms.eaSimple(pop, toolbox, cxpb=0.9, mutpb=0.3, ngen=
        ga_params['ngen'], halloffame=hof, verbose=False)
117
118     best_splits = sorted(hof[0])
119     boundaries = [-np.inf] + best_splits + [np.inf]
120
121     for i in range(len(boundaries) - 1):
122         bmi_min, bmi_max = boundaries[i], boundaries[i+1]
123         group_df = df_valid[(df_valid['平均BMI'] > bmi_min) & (
            df_valid['平均BMI'] <= bmi_max)]
124         if not group_df.empty:
125             bmi_range_str = f"<= {bmi_max:.2f}" if i == 0 else f"
                > {bmi_min:.2f}" if i == len(boundaries) - 2 else
                f"{bmi_min:.2f} - {bmi_max:.2f}"
126             results.append({
127                 'accuracy': p, 'group_index': i, 'bmi_range':
                    bmi_range_str,
128                 'group_size': len(group_df), '
                    recommended_nipt_week': group_df['T_attain'].
                    quantile(p / 100.0)
129             })
130     print("第二步: 策略优化完成 ---")
131     return pd.DataFrame(results)
132
133 # 模块三: 结果可视化
134 def plot_final_strategy(df_results, output_path='
    problem2_final_strategy.html'):
135
136     print("第三步: 开始生成可视化图表")
137     fig = go.Figure()
138
139     fig.add_hrect(y0=0, y1=12, line_width=0, fillcolor='rgba(44, 160,
        44, 0.06)')
140     fig.add_hrect(y0=12, y1=28, line_width=0, fillcolor='rgba(255,
        127, 14, 0.06)')
141     fig.add_hrect(y0=28, y1=df_results['recommended_nipt_week'].max()
        + 2, line_width=0, fillcolor='rgba(214, 39, 40, 0.06)')
142
143     colors = ['rgba(44, 160, 44, 1.0)', 'rgba(31, 119, 180, 1.0)', '
        rgba(214, 39, 40, 1.0)']
144     group_names = {0: "低BMI组", 1: "中BMI组", 2: "高BMI组"}
145     group_markers = ['circle', 'square', 'diamond']
146
147     plot_order = [2, 1, 0]
148     for i in plot_order:

```



```

149     group_df = df_results[df_results['group_index'] == i]
150     if not group_df.empty:
151         fill_mode = 'tonexty' if i > 0 else 'tozero'
152         fig.add_trace(go.Scatter(
153             x=group_df['accuracy'], y=group_df['
154                 recommended_nipt_week'], name=group_names[i],
155             mode='lines+markers', line=dict(width=2.1, color=
156                 colors[i]),
157             marker=dict(symbol=group_markers[i], size=6, line=
158                 dict(width=0)),
159             fill=fill_mode, fillcolor=colors[i].replace('1.0', '
160                 0.1'),
161             hoverinfo='text',
162             hovertext=[f"<b>准确率:</b> {r['accuracy']}%<br><b>分
163                 组:</b> {group_names[i]}<br><b>BMI范围:</b> {r['
164                 bmi_range']}<br><b>孕周:</b> {r['
165                 recommended_nipt_week']:.2f}" for _, r in group_df
166                 .iterrows()]
167         ))
168
169     fig.update_layout(
170         title=dict(text="NIPT策略自适应优化模型 (基于GPR+GA)", y
171             =0.96, x=0.5, font=dict(size=24, family="Times New Roman,
172             SimSun")),
173         xaxis_title="准确率要求 (%)", yaxis_title="推荐NIPT孕周",
174         xaxis=dict(showgrid=True, gridwidth=1, gridcolor='rgba
175             (220,220,220,0.5)', showline=True, linewidth=1, linecolor=
176             'black', mirror=True, ticks='outside', range=[49, 101]),
177         yaxis=dict(showgrid=True, gridwidth=1, gridcolor='rgba
178             (220,220,220,0.5)', showline=True, linewidth=1, linecolor=
179             'black', mirror=True, ticks='outside'),
180         legend=dict(x=0.03, y=0.97, yanchor="top", xanchor="left",
181             bgcolor='rgba(255, 255, 255, 0.8)', bordercolor="grey",
182             borderwidth=0.5),
183         plot_bgcolor='white', paper_bgcolor='white', hovermode='x
184             unified',
185         font=dict(family="Times New Roman, SimSun", size=13, color="
186             black"),
187         margin=dict(l=100, r=40, b=80, t=120), width=1200, height=800
188     )
189
190     fig.write_html(output_path)
191     print(f"第三步: 策略图已保存至: {output_path} ---")
192
193 # 主执行流程
194 if __name__ == '__main__':
195     RAW_DATA_PATH = 'data_preprocessed_final.xlsx'
196     T_ATTAIN_OUTPUT_PATH = 'problem2_t_attain_predicted.csv'
197     OPTIMIZATION_RESULTS_PATH = 'problem2_optimization_results.csv'
198     FINAL_PLOT_PATH = 'problem2_final_strategy.html'
199
200     GA_PARAMS = {'num_splits': 2, 'pop_size': 100, 'ngen': 50, 'p_min
201         ': 50, 'p_max': 100}
202
203     print("开始执行问题二: GPR预测 + GA优化 + 可视化")
204
205     # 模块一: 计算或加载 T_attain

```

```

187     if os.path.exists(T_ATTAIN_OUTPUT_PATH):
188         print(f"检测到已存在的 T_attain 文件, 直接加载")
189         df_t_attain = pd.read_csv(T_ATTAIN_OUTPUT_PATH)
190     else:
191         df_raw = pd.read_excel(RAW_DATA_PATH)
192         df_male = df_raw[df_raw['Y染色体浓度'].notna() & (df_raw['Y染色体浓度'] > 0)]
193         df_t_attain = calculate_t_attain(df_male)
194         df_t_attain.to_csv(T_ATTAIN_OUTPUT_PATH, index=False,
195                             encoding='utf-8-sig')
196         print(f"预测的T_attain结果已保存至: {T_ATTAIN_OUTPUT_PATH}")
197
198     # 模块二: 优化或加载结果
199     if os.path.exists(OPTIMIZATION_RESULTS_PATH):
200         print(f"检测到已存在的优化结果文件, 直接加载 ---")
201         results_df = pd.read_csv(OPTIMIZATION_RESULTS_PATH)
202     else:
203         df_valid_for_opt = df_t_attain[np.isfinite(df_t_attain['
204             T_attain'])].copy()
205         results_df = run_genetic_algorithm(df_valid_for_opt,
206             GA_PARAMS)
207         results_df.to_csv(OPTIMIZATION_RESULTS_PATH, index=False,
208                             encoding='utf-8-sig')
209         print(f"\n优化结果已保存至: {OPTIMIZATION_RESULTS_PATH}")
210
211     # 模块三: 可视化
212     if not results_df.empty:
213         plot_final_strategy(results_df, FINAL_PLOT_PATH)
214
215     print("问题二已完成")

```

Listing 附录 B.3 问题 3: 求解 BMI 分组和最佳 NIPT 时点的 Python 代码 (problem3.py)

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.model_selection import train_test_split
5 import pickle
6 import torch
7 import torch.nn as nn
8 from pycox.models import DeepHitSingle
9 import matplotlib.pyplot as plt
10 from tqdm import tqdm
11 import random
12 from deap import base, creator, tools, algorithms
13 import plotly.graph_objects as go
14 import warnings
15 import os
16
17 warnings.filterwarnings("ignore", category=UserWarning)
18 plt.rcParams['font.sans-serif'] = ['SimHei']
19 plt.rcParams['axes.unicode_minus'] = False
20
21 # 第一步: 数据准备 & 模型训练 & 特征重要性
22
23 def prepare_data_for_survival_model(df_raw, t_attain_path):
24     print("开始执行数据准备与特征工程")

```

```

25 df_t_attain = pd.read_csv(t_attain_path)
26 df_raw.rename(columns={'孕妇代码': 'Person_id'}, inplace=True)
27
28 static_features = df_raw.sort_values('检测孕周_数值').groupby('
    Person_id').first()
29 static_features = static_features[['年龄', '身高', '体重', '孕妇
    BMI']].rename(columns={
30     '体重': 'Initial_Weight', '孕妇BMI': 'Initial_BMI', '年龄': '
        Age', '身高': 'Height'
31 })
32 longitudinal_features = {pid: list(zip(g.sort_values('检测孕周_数
    值')['检测孕周_数值'], g.sort_values('检测孕周_数值')['Y染色体
    浓度'])) for pid, g in df_raw.groupby('Person_id')}
33
34 df_survival = df_t_attain[['Person_id', 'T_attain']].copy()
35 df_survival['E'] = ~np.isinf(df_survival['T_attain'])
36 last_obs = df_raw.groupby('Person_id')['检测孕周_数值'].max().
    to_dict()
37 df_survival['T'] = df_survival.apply(lambda r: r['T_attain'] if r
    ['E'] else last_obs.get(r['Person_id']), axis=1)
38 df_survival.dropna(subset=['T'], inplace=True)
39 df_survival['E'] = df_survival['E'].astype(int)
40
41 df_final = static_features.join(df_survival.set_index('Person_id'
    ), how='inner')
42 df_final['longitudinal'] = df_final.index.map(
    longitudinal_features)
43 df_final.reset_index(inplace=True)
44
45 static_scaler = StandardScaler().fit(df_final[['Age', 'Height', '
    Initial_Weight', 'Initial_BMI']])
46 df_final[['Age', 'Height', 'Initial_Weight', 'Initial_BMI']] =
    static_scaler.transform(df_final[['Age', 'Height', '
    Initial_Weight', 'Initial_BMI']])
47
48 max_len = df_final['longitudinal'].apply(len).max()
49 all_conc = [item[1] for sublist in df_final['longitudinal'] for
    item in sublist]
50 long_scaler = StandardScaler().fit(np.array(all_conc).reshape(-1,
    1))
51
52 padded_seqs, masks = [], []
53 for seq in df_final['longitudinal']:
54     norm_seq = [(item[0], long_scaler.transform([[item[1]]])
55         [0,0]) for item in seq]
56     padded_seqs.append(norm_seq + [(0, 0)] * (max_len - len(
57         norm_seq)))
58     masks.append([1] * len(norm_seq) + [0] * (max_len - len(
59         norm_seq)))
60
61 df_final['longitudinal_padded'] = padded_seqs
62 df_final['mask'] = masks
63 df_final['T_discrete'] = np.floor(df_final['T']).astype(int)
64
65 print(f"数据处理完成")
66 return df_final, static_scaler, long_scaler

```

```

65 class HybridDeepSurvNet(nn.Module):
66     def __init__(self, in_static, in_long, lstm_hidden, mlp_hidden,
67                 out_features, dropout):
68         super().__init__()
69         self.mlp_encoder = nn.Sequential(*[layer for dim_in, dim_out
70         in zip([in_static] + mlp_hidden[:-1], mlp_hidden) for
71         layer in (nn.Linear(dim_in, dim_out), nn.ReLU(), nn.
72         Dropout(dropout))])
73         self.lstm_encoder = nn.LSTM(in_long, lstm_hidden, batch_first
74         =True)
75         self.fusion_layer = nn.Sequential(nn.Linear(mlp_hidden[-1] +
76         lstm_hidden, out_features), nn.ReLU(), nn.Dropout(dropout)
77         )
78     def forward(self, x_static, x_long, mask):
79         h_static = self.mlp_encoder(x_static)
80         _, (h_long, _) = self.lstm_encoder(x_long)
81         h_fused = torch.cat([h_static, h_long.squeeze(0)], dim=1)
82         return self.fusion_layer(h_fused)
83
84 def train_survival_model(df_processed):
85     print("开始执行模型构建与训练")
86     static = torch.tensor(df_processed[['Age', 'Height', '
87     Initial_Weight', 'Initial_BMI']].values, dtype=torch.float32)
88     long = torch.tensor(np.array(df_processed['longitudinal_padded'].
89     tolist()), dtype=torch.float32)
90     masks = torch.tensor(np.array(df_processed['mask'].tolist()),
91     dtype=torch.float32)
92     durations = torch.tensor(df_processed['T_discrete'].values).long
93     ()
94     events = torch.tensor(df_processed['E'].values).float()
95
96     train_idx, val_idx = train_test_split(np.arange(len(df_processed)
97     ), test_size=0.2, random_state=42)
98     x_train = (static[train_idx], long[train_idx], masks[train_idx])
99     y_train = (durations[train_idx], events[train_idx])
100     x_val = (static[val_idx], long[val_idx], masks[val_idx])
101     y_val = (durations[val_idx], events[val_idx])
102
103     labtrans = DeepHitSingle.label_transform(np.arange(durations.max
104     () + 1))
105     net = HybridDeepSurvNet(4, 2, 16, [32, 16], labtrans.out_features
106     , 0.4)
107     model = DeepHitSingle(net, duration_index=labtrans.cuts, alpha
108     =0.2, sigma=0.1)
109
110     model.optimizer = torch.optim.Adam(model.net.parameters(), lr=1e
111     -3, weight_decay=1e-4)
112     log = model.fit(x_train, y_train, 32, 200, val_data=(x_val, y_val
113     ), verbose=False)
114
115     log.plot()
116     plt.savefig('problem3_training_loss_curve.png')
117     plt.close()
118     print(f"模型训练完成，损失曲线已保存")
119
120     return model.net, x_val, y_val, labtrans

```



```

105 def plot_feature_importance(net, x_val, y_val, labtrans):
106     print("开始执行特征重要性分析")
107     model = DeepHitSingle(net, duration_index=labtrans.cuts)
108     device = next(net.parameters()).device
109     x_val = tuple(x.to(device) for x in x_val)
110     y_val = tuple(y.to(device) for y in y_val)
111     model.net.eval()
112
113     base_loss = model.score_in_batches(x_val, y_val, batch_size=len(
        y_val[0]))['loss']
114
115     importances = {}
116     feature_names = {0: '年龄', 1: '身高', 2: '初始体重', 3: '初始BMI', 'long': '时序信息'}
117     static_val, long_val, mask_val = x_val
118
119     for col_idx in tqdm(range(static_val.shape[1]), desc="评估静态特征"):
120         static_permuted = static_val.clone()
121         perm = torch.randperm(static_permuted.size(0))
122         static_permuted[:, col_idx] = static_permuted[perm, col_idx]
123         loss = model.score_in_batches((static_permuted, long_val,
            mask_val), y_val, batch_size=len(y_val[0]))['loss']
124         importances[feature_names[col_idx]] = loss - base_loss
125
126     long_permuted = long_val.clone()[torch.randperm(long_val.size(0))]
127     loss = model.score_in_batches((static_val, long_permuted,
        mask_val), y_val, batch_size=len(y_val[0]))['loss']
128     importances[feature_names['long']] = loss - base_loss
129
130     # 绘制包含所有特征的重要性图
131     sortedimps = sorted(importances.items(), key=lambda item: item
        [1])
132     names, values = zip(*sortedimps)
133     plt.figure(figsize=(12, 8)); bars = plt.barh(names, values, color
        ='skyblue'); plt.xlabel('特征重要性 (验证集损失增加量)'); plt.
        title('多因素对NIPT达标时间预测的重要性分析')
134     for bar in bars: plt.text(bar.get_width(), bar.get_y()+bar.
        get_height()/2, f' {bar.get_width():.4f}', va='center')
135     plt.savefig('problem3_all_feature_importance.png', dpi=300,
        bbox_inches='tight'); plt.close()
136
137     # 绘制仅包含静态特征的重要性图
138     staticimps = {k: v for k, v in importances.items() if k != '时序
        信息'}
139     sorted_staticimps = sorted(staticimps.items(), key=lambda item:
        item[1])
140     names, values = zip(*sorted_staticimps)
141     plt.figure(figsize=(10, 6)); bars = plt.barh(names, values, color
        ='dodgerblue'); plt.xlabel('特征重要性 (验证集损失增加量)');
        plt.title('静态生理特征对NIPT达标时间预测的重要性分析')
142     for bar in bars: plt.text(bar.get_width(), bar.get_y()+bar.
        get_height()/2, f' {bar.get_width():.4f}', va='center')
143     plt.savefig('problem3_static_feature_importance.png', dpi=300,
        bbox_inches='tight'); plt.close()
144     print(f"特征重要性分析完成, 图表已保存")

```

```

145
146 # 第二步：基于模型的策略优化与可视化
147 class SurvivalOracle:
148     def __init__(self, model, df_processed):
149         self.model = model
150         self.df_full = df_processed
151         self.model.net.eval()
152     def predict_surv_for_group(self, person_ids):
153         if not person_ids: return None
154         group_df = self.df_full[self.df_full['Person_id'].isin(
            person_ids)]
155         if group_df.empty: return None
156         static = torch.tensor(group_df[['Age', 'Height', '
            Initial_Weight', 'Initial_BMI']].values, dtype=torch.
            float32)
157         long = torch.tensor(np.array(group_df['longitudinal_padded'].
            tolist()), dtype=torch.float32)
158         mask = torch.ones_like(long[:, :, 0])
159         with torch.no_grad():
160             surv_df = self.model.predict_surv_df((static, long, mask)
            )
161         return surv_df.mean(axis=1)
162
163 def get_daily_interpolated_week(prob_curve, p_target):
164     if prob_curve.iloc[0] >= p_target: return prob_curve.index[0]
165     above = prob_curve[prob_curve >= p_target]
166     if above.empty: return prob_curve.index.max()
167     t2, p2 = above.index.min(), above.iloc[0]
168     below = prob_curve[prob_curve < p_target]
169     if below.empty: return t2
170     t1, p1 = below.index.max(), below.iloc[-1]
171     if p2 <= p1: return t2
172     daily_increase = (p2 - p1) / 7.0
173     for day in range(1, 8):
174         if p1 + day * daily_increase >= p_target: return t1 + day /
            7.0
175     return t2
176
177 class GeneticOptimizer:
178     def __init__(self, oracle, df_raw, ga_params):
179         self.oracle = oracle; self.ga_params = ga_params
180         df_bmi_map = df_raw.sort_values('检测孕周_数值').groupby('
            Person_id').first().reset_index()
181         self.df_bmi_map = df_bmi_map.rename(columns=('孕妇BMI': '
            Initial_BMI_orig'))
182         creator.create("FitnessMin", base.Fitness, weights=(-1.0,));
            creator.create("Individual", list, fitness=creator.
            FitnessMin)
183         min_bmi, max_bmi = self.df_bmi_map['Initial_BMI_orig'].min(),
            self.df_bmi_map['Initial_BMI_orig'].max()
184         self.toolbox = base.Toolbox(); self.toolbox.register("
            attr_float", random.uniform, min_bmi, max_bmi)
185         self.toolbox.register("individual", tools.initRepeat, creator
            .Individual, self.toolbox.attr_float, ga_params['
            num_splits'])
186         self.toolbox.register("population", tools.initRepeat, list,
            self.toolbox.individual); self.toolbox.register("mate",

```



```

tools.cxBlend, alpha=0.5)
187 self.toolbox.register("mutate", tools.mutGaussian, mu=0,
    sigma=(max_bmi-min_bmi)*0.1, indpb=0.2); self.toolbox.
    register("select", tools.selTournament, tournsize=3)
188 def evaluate(self, individual, p_target):
189     splits = sorted(individual); groups = []; last_b = -np.inf
190     for b in splits:
191         groups.append(self.df_bmi_map[(self.df_bmi_map['
            Initial_BMI_orig'] > last_b) & (self.df_bmi_map['
            Initial_BMI_orig'] <= b)]['Person_id'].tolist());
            last_b = b
192     groups.append(self.df_bmi_map[self.df_bmi_map['
        Initial_BMI_orig'] > last_b]['Person_id'].tolist())
193     total_risk = 0
194     for group_ids in groups:
195         if len(group_ids) < self.ga_params['min_group_size']:
            return (1e9 + (self.ga_params['min_group_size'] - len(
                group_ids)) * 1000,)
196         surv = self.oracle.predict_surv_for_group(group_ids)
197         if surv is None: continue
198         week = get_daily_interpolated_week(1 - surv, p_target)
199         total_risk += (1 if week <= 12 else 5 if week <= 28 else
            10) * len(group_ids)
200     return (total_risk,)
201 def run(self):
202     print("开始执行遗传算法策略优化"); all_results = []
203     for p in tqdm(range(self.ga_params['p_min'], self.ga_params['
        p_max']), desc="策略优化进度 (基于深度模型)":
204         p_target = p / 100.0; self.toolbox.register("evaluate",
            self.evaluate, p_target=p_target)
205         pop = self.toolbox.population(n=self.ga_params['pop_size']
            ); hof = tools.HallOfFame(1)
206         algorithms.eaSimple(pop, self.toolbox, cxpb=0.7, mutpb
            =0.2, ngen=self.ga_params['ngen'], halloffame=hof,
            verbose=False)
207         best_splits = sorted(hof[0]); boundaries = [-np.inf] +
            best_splits + [np.inf]
208         for i in range(len(boundaries) - 1):
209             b_start, b_end = boundaries[i], boundaries[i+1]
210             group_ids = self.df_bmi_map[(self.df_bmi_map['
                Initial_BMI_orig'] > b_start) & (self.df_bmi_map['
                Initial_BMI_orig'] <= b_end)]['Person_id'].tolist
                ()
211             if not group_ids: continue
212             week = get_daily_interpolated_week(1 - self.oracle.
                predict_surv_for_group(group_ids), p_target)
213             all_results.append({'accuracy': p, 'group_index': i,
                'bmi_range': f"{b_start:.2f} - {b_end:.2f}", '
                recommended_nipt_week': week, 'group_size': len(
                group_ids)})
214         print("策略优化完成")
215         return pd.DataFrame(all_results)
216
217 def plot_dynamic_strategy_visuals(df_results, output_html='
    problem3_strategy_dynamic.html'):
218     print("开始生成策略可视化图表")
219     fig = go.Figure()

```

```

220 sample_df = df_results[df_results['accuracy'] == df_results['
    accuracy'].min()].copy()
221 sample_df['bmi_min'] = sample_df['bmi_range'].apply(lambda x:
    float(x.split(' - ')[0]))
222
223 group_map = {row.group_index: name for name, row in zip(["低BMI组",
    "中BMI组", "高BMI组"], sample_df.sort_values('bmi_min').
    itertuples())}
224 df_results['group_name'] = df_results['group_index'].map(
    group_map)
225
226 colors = ['rgba(44, 160, 44, 1.0)', 'rgba(31, 119, 180, 1.0)', '
    rgba(214, 39, 40, 1.0)']
227 order = ["低BMI组", "中BMI组", "高BMI组"]; markers = ['circle', '
    square', 'diamond']
228
229 for i, name in enumerate(order):
230     item_df = df_results[df_results['group_name'] == name]
231     fig.add_trace(go.Scatter(x=item_df['accuracy'], y=item_df['
        recommended_nipt_week'], name=name, mode='lines+markers',
        line=dict(color=colors[i]), marker=dict(symbol=markers[i])
        ))
232
233 fig.update_layout(title="<b>基于深度学习的多因素NIPT策略优化模型
    </b>", xaxis_title="Y染色体浓度达标比例 (%)", yaxis_title="推
    荐NIPT孕周")
234 fig.write_html(output_html)
235 print(f"可视化完成，动态图表已保存至：{output_html}")
236
237 # 主执行流程
238 if __name__ == '__main__':
239     RAW_DATA_PATH = 'data_preprocessed_final.xlsx'
240     T_ATTAIN_PATH = 'problem2_t_attain_predicted.csv'
241     OPTIMIZATION_RESULTS_PATH = 'problem3_optimization_results.csv'
242
243     GA_PARAMS = {'num_splits': 2, 'min_group_size': 50, 'pop_size':
        50, 'ngen': 100, 'p_min': 50, 'p_max': 100}
244
245     print("开始执行问题三完整流程")
246
247     if os.path.exists(OPTIMIZATION_RESULTS_PATH):
248         print(f"--- 检测到已存在的优化结果文件 {
            OPTIMIZATION_RESULTS_PATH}, 直接加载并进行可视化 ---")
249         results_df = pd.read_csv(OPTIMIZATION_RESULTS_PATH)
250     else:
251         try:
252             pd.read_csv(T_ATTAIN_PATH)
253         except FileNotFoundError:
254             print(f"错误：未找到T_attain预测文件 '{T_ATTAIN_PATH}'。
                请先成功运行 problem2.py。"); exit()
255
256         df_raw_male = pd.read_excel(RAW_DATA_PATH)
257         df_raw_male = df_raw_male[df_raw_male['Y染色体浓度'].notna()
            & (df_raw_male['Y染色体浓度'] > 0)]
258
259         df_processed, static_scaler, long_scaler =
            prepare_data_for_survival_model(df_raw_male, T_ATTAIN_PATH

```

```

    )
260
261     net, x_val, y_val, labtrans = train_survival_model(
        df_processed)
262
263     plot_feature_importance(net, x_val, y_val, labtrans)
264
265     oracle_model = DeepHitSingle(net, duration_index=labtrans.
        cuts)
266     oracle = SurvivalOracle(oracle_model, df_processed)
267     optimizer = GeneticOptimizer(oracle, df_raw_male, GA_PARAMS)
268     results_df = optimizer.run()
269     results_df.to_csv(OPTIMIZATION_RESULTS_PATH, index=False,
        encoding='utf-8-sig')
270     print(f"\n优化结果已保存至: {OPTIMIZATION_RESULTS_PATH}")
271
272     plot_dynamic_strategy_visuals(results_df)
273
274     print("问题三所有任务已完成")

```

Listing 附录 B.4 问题 4: 女胎异常评定方法的 Python 代码 (problem4.py)

```

1
2 import argparse
3 import pickle
4 import sys
5 import warnings
6 from pathlib import Path
7
8 import lightgbm as lgb
9 import matplotlib
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import optuna
13 import pandas as pd
14 import seaborn as sns
15 from imblearn.over_sampling import SMOTE
16 from sklearn.feature_selection import RFE
17 from sklearn.metrics import (classification_report, confusion_matrix,
18                             log_loss)
19 from sklearn.model_selection import StratifiedKFold
20 from sklearn.preprocessing import StandardScaler
21
22 warnings.filterwarnings('ignore')
23 matplotlib.rcParams['font.sans-serif'] = ['SimHei']
24 matplotlib.rcParams['axes.unicode_minus'] = False
25
26 # 1. 配置模块
27 # 所有文件都将与此脚本位于同一目录中
28 BASE_DIR = Path(__file__).resolve().parent
29
30 # 原始数据文件直接位于脚本所在目录
31 RAW_DATA_FILE_1 = BASE_DIR / "data.xlsx"
32 RAW_DATA_FILE_2 = BASE_DIR / "data_preprocessed_final.xlsx"
33 # 处理后的数据文件也直接位于脚本所在目录
34 MODELING_DATA_FILE = BASE_DIR / "data_q4_modeling.csv"
35

```

```

36 # 为了代码兼容性，保留变量名，但都指向BASE_DIR
37 DATA_DIR = BASE_DIR
38 RAW_DATA_DIR = BASE_DIR
39 PROCESSED_DATA_DIR = BASE_DIR
40 OUTPUTS_DIR = BASE_DIR
41 LOGS_DIR = BASE_DIR
42 IMAGES_DIR = BASE_DIR
43 OPTUNA_STUDIES_DIR = BASE_DIR
44 OPTUNA_DIR = BASE_DIR
45 MODELS_DIR = BASE_DIR
46
47 # --- 模型与特征相关的常量 ---
48 TARGET_COLUMNS = ["is_abnormal_13", "is_abnormal_18", "is_abnormal_21"]
49 INTERACTION_TARGET = "is_multi_abnormal"
50
51 EXPERT_MODELS_CONFIG = {
52     "t13": {"name": "t13_expert_model.pkl", "target": "is_abnormal_13"},
53     "t18": {"name": "t18_expert_model.pkl", "target": "is_abnormal_18"},
54     "t21": {"name": "t21_expert_model.pkl", "target": "is_abnormal_21"},
55     "interaction": {"name": "interaction_expert_model.pkl", "target": INTERACTION_TARGET}
56 }
57
58 META_MODEL_NAME = "final_meta_model.pkl"
59
60 CORE_MEDICAL_FEATURES = [
61     '13号染色体的Z值_df1', '18号染色体的Z值_df1', '21号染色体的Z值_df1',
62     'X染色体的Z值_df1', 'Y染色体的Z值_df1', 'GC含量_df1',
63     '13号染色体的GC含量_df1', '18号染色体的GC含量_df1', '21号染色体的GC含量_df1',
64     '唯一比对的读段数_df1', '被过滤掉读段数的比例_df1', '孕妇BMI_df1'
65 ]
66
67 KEY_ORIGINAL_FEATURES = [
68     '年龄_df1', '孕妇BMI_df1',
69     '13号染色体的Z值_df1', '18号染色体的Z值_df1', '21号染色体的Z值_df1'
70 ]
71
72 # --- 最终评估相关的常量 ---
73 FINAL_CONFUSION_MATRIX_PATH = IMAGES_DIR / "logic_enhanced_confusion_matrix.png"
74
75 STATE_LABELS = {
76     0: "健康", 1: "仅T13", 2: "仅T18", 3: "T13+T18",
77     4: "仅T21", 5: "T13+T21", 6: "T18+T21", 7: "T13+T18+T21"
78 }
79
80 Z_SCORE_THRESHOLD = 3.0
81
82 # 2. 数据预处理模块
83 def _parse_gestational_week(week_str):

```



```

84     """将孕周字符串解析为浮点数。"""
85     try:
86         week_str = str(week_str).lower()
87         if 'w+' in week_str:
88             w, d = week_str.split('w+')
89             return int(w) + int(d) / 7.0
90         elif 'w' in week_str:
91             return int(week_str.replace('w', ''))
92         return float(week_str)
93     except (ValueError, TypeError):
94         return np.nan
95
96 def create_modeling_data():
97     print("开始数据预处理流程")
98     try:
99         print(f"正在从 '{RAW_DATA_FILE_1}' 和 '{RAW_DATA_FILE_2}' 加
100             载数据...")
101         df1 = pd.read_excel(RAW_DATA_FILE_1, sheet_name=1)
102         df2 = pd.read_excel(RAW_DATA_FILE_2)
103
104         df1.columns = df1.columns.str.strip()
105         df2.columns = df2.columns.str.strip()
106
107         df = pd.merge(df1, df2, on="孕妇代码", how="left", suffixes=(
108             '_df1', None))
109         print(f"数据加载与合并完成。初始维度: {df.shape}")
110
111         print("步骤 1: 创建目标变量")
112         abnormal_col = '染色体的非整倍体_df1'
113         df['is_abnormal_13'] = df[abnormal_col].apply(lambda x: 1 if
114             'T13' in str(x) else 0)
115         df['is_abnormal_18'] = df[abnormal_col].apply(lambda x: 1 if
116             'T18' in str(x) else 0)
117         df['is_abnormal_21'] = df[abnormal_col].apply(lambda x: 1 if
118             'T21' in str(x) else 0)
119         df[INTERACTION_TARGET] = ((df['is_abnormal_13'] + df['
120             is_abnormal_18'] + df['is_abnormal_21']) >= 2).astype(int)
121
122         feature_cols = [
123             col for col in df.columns
124             if col not in ['孕妇代码', '染色体的非整倍体'] +
125                 TARGET_COLUMNS + [INTERACTION_TARGET]
126         ]
127
128         print("步骤 2: 清洗特定文本特征")
129         if 'IVF妊娠' in df.columns:
130             df['IVF妊娠'] = df['IVF妊娠'].apply(lambda x: 0 if '自然'
131                 in str(x) else 1)
132         if '检测孕周' in df.columns:
133             df['检测孕周'] = df['检测孕周'].apply(
134                 _parse_gestational_week)
135
136         print("步骤 2.5: 对年龄、身高、体重进行标准化")
137         scaler = StandardScaler()
138         features_to_scale = ['年龄_df1', '身高_df1', '体重_df1']
139
140         for col in features_to_scale:

```

```

132         df[col] = pd.to_numeric(df[col], errors='coerce')
133         df[col].fillna(df[col].median(), inplace=True)
134         df[f'{col}_scaled'] = scaler.fit_transform(df[[col]])
135
136     for col in features_to_scale:
137         if col in feature_cols:
138             feature_cols.remove(col)
139             feature_cols.append(f'{col}_scaled')
140
141     print("步骤 3: 强制所有特征列转为数值类型")
142     for col in feature_cols:
143         df[col] = pd.to_numeric(df[col], errors='coerce')
144         if df[col].isnull().all():
145             df.drop(columns=[col], inplace=True)
146             print(f"已删除纯文本列: '{col}'")
147
148     print("步骤 4: 使用中位数填充缺失值")
149     df.fillna(df.median(numeric_only=True), inplace=True)
150
151     final_columns = [
152         col for col in df.columns
153         if col in feature_cols + TARGET_COLUMNS + [
154             INTERACTION_TARGET] and df[col].dtype in [np.int64, np
155             .float64, np.int32]
156     ]
157     df_final = df[final_columns]
158
159     non_feature_cols = ['序号_df1']
160     df_final = df_final.drop(columns=[col for col in
161         non_feature_cols if col in df_final.columns], errors='
162         ignore')
163     print(f"已明确剔除非预测性特征: {non_feature_cols}")
164
165     MODELING_DATA_FILE.parent.mkdir(parents=True, exist_ok=True)
166     df_final.to_csv(MODELING_DATA_FILE, index=False, encoding='
167         utf-8-sig')
168
169     print("\n数据预处理流程成功完成")
170     print(f"最终建模文件已保存至: '{MODELING_DATA_FILE}'")
171     print(f"最终数据维度: {df_final.shape}")
172
173     return df_final
174
175 except FileNotFoundError as e:
176     print(f"错误: 找不到原始数据文件。 {e}")
177     return None
178 except Exception as e:
179     print(f"在数据预处理过程中发生未知错误: {e}")
180     return None
181
182 # 3. 模型训练模块
183 def _train_single_expert(X, y, study_name, n_trials=50):
184     """ 使用Optuna训练并优化单个二分类专家模型。 """
185     print(f"开始为目标 '{y.name}' 训练专家模型")
186
187     def objective(trial):
188         params = {

```



```

184         'objective': 'binary', 'metric': 'logloss', 'verbosity':
185             -1,
186         'is_unbalance': True, 'n_estimators': 1000,
187         'learning_rate': trial.suggest_float('learning_rate',
188             0.01, 0.1),
189         'num_leaves': trial.suggest_int('num_leaves', 10, 150),
190     }
191     skf = StratifiedKFold(n_splits=5, shuffle=True, random_state
192         =42)
193     logloss_scores = []
194     for train_index, val_index in skf.split(X, y):
195         model = lgb.LGBMClassifier(**params)
196         model.fit(X.iloc[train_index], y.iloc[train_index],
197             eval_set=[(X.iloc[val_index], y.iloc[val_index
198                 ])],
199             eval_metric='logloss',
200             callbacks=[lgb.early_stopping(50, verbose=False
201                 )])
202
203         y_val = y.iloc[val_index]
204         if len(np.unique(y_val)) < 2:
205             print(f"在为 '{y.name}' 进行交叉验证时，其中一折只有一个
206                 类别，跳过此折评估。")
207             continue
208
209         preds = model.predict_proba(X.iloc[val_index])[:, 1]
210         logloss_scores.append(log_loss(y_val, preds))
211
212     return np.mean(logloss_scores) if logloss_scores else 999.0
213
214 storage_path = f"sqlite:///({OPTUNA_STUDIES_DIR} / f'optuna_{
215     study_name}.db' )"
216 study = optuna.create_study(direction='minimize', study_name=
217     study_name, storage=storage_path, load_if_exists=True)
218 study.optimize(objective, n_trials=n_trials, show_progress_bar=
219     True)
220
221 print(f"为 '{y.name}' 找到的最佳LogLoss: {study.best_value:.6f}")
222
223 final_model = lgb.LGBMClassifier(**study.best_params)
224 final_model.fit(X, y)
225
226 return final_model
227
228 def train_all_expert_models():
229     """训练并保存所有在config中定义的专家模型。"""
230     print("--- 开始训练所有专家模型 ---")
231     df = pd.read_csv(MODELING_DATA_FILE)
232
233     all_targets = TARGET_COLUMNS + [INTERACTION_TARGET]
234     X = df.drop(columns=[col for col in all_targets if col in df.
235         columns])
236
237     for model_key, config in EXPERT_MODELS_CONFIG.items():
238         target_col = config['target']
239         if target_col not in df.columns:
240             print(f"在数据集中找不到目标列 '{target_col}'。跳过模型

```

```

        '{model_key}'.")
231     continue
232
233     y = df[target_col]
234     study_name_for_optuna = f"expert_{target_col}"
235     model = _train_single_expert(X, y, study_name=
        study_name_for_optuna, n_trials=50)
236
237     save_path = MODELS_DIR / config['name']
238     with open(save_path, 'wb') as f:
239         pickle.dump(model, f)
240     print(f"专家模型 '{config['name']}' 已保存至 '{save_path}'\n"
        )
241
242 def train_meta_model(n_trials=100, n_splits=5):
243     """通过健壮的K折交叉验证流程训练元模型。"""
244     print(f"使用{n_splits}折交叉验证流程训练元模型")
245     df = pd.read_csv(MODELING_DATA_FILE)
246
247     print("步骤 1: 使用交叉验证生成无泄露的元特征")
248
249     all_targets = TARGET_COLUMNS + [INTERACTION_TARGET]
250     X_original = df.drop(columns=[col for col in all_targets if col
        in df.columns], errors='ignore')
251     y_multistate = df[TARGET_COLUMNS[2]] * 4 + df[TARGET_COLUMNS[1]]
        * 2 + df[TARGET_COLUMNS[0]] * 1
252
253     meta_features = pd.DataFrame(index=X_original.index)
254     skf = StratifiedKFold(n_splits=n_splits, shuffle=True,
        random_state=42)
255
256     for model_key, config in EXPERT_MODELS_CONFIG.items():
257         target_col = config['target']
258         print(f"正在为 '{target_col}' 生成元特征")
259
260         study_name = f"expert_{target_col}"
261         storage_path = f"sqlite:///({OPTUNA_STUDIES_DIR / f'optuna_{
        study_name}.db'})"
262         try:
263             study = optuna.load_study(study_name=study_name, storage=
        storage_path)
264             best_params = study.best_params
265             print(f"> 成功从 '{study_name}' 加载最佳参数。")
266         except Exception as e:
267             print(f"> 警告: 无法加载 '{study_name}' 的Optuna研
        究, 将使用默认LGBM参数。错误: {e}")
268             best_params = {}
269
270         oof_preds = np.zeros(len(df))
271         X_expert, y_expert = X_original.copy(), df[target_col]
272
273         for train_index, val_index in skf.split(X_expert, y_expert):
274             X_train, X_val = X_expert.iloc[train_index], X_expert.
        iloc[val_index]
275             y_train = y_expert.iloc[train_index]
276
277             temp_model = lgb.LGBMClassifier(**best_params)

```



```

278         temp_model.fit(X_train, y_train)
279         oof_preds[val_index] = temp_model.predict_proba(X_val)[: ,
280             1]
281
282         meta_features[f"{target_col}_prob"] = oof_preds
283
284     print("步骤 2: 混合策略特征选择")
285
286     forced_features = list(meta_features.columns)
287     existing_core_features = [f for f in CORE_MEDICAL_FEATURES if f
288         in X_original.columns]
289     forced_features.extend(existing_core_features)
290
291     remaining_features = [f for f in X_original.columns if f not in
292         existing_core_features]
293
294     print(f" 强制保留特征: {len(forced_features)} 个")
295     print(f" 待RFE探索特征: {len(remaining_features)} 个")
296
297     selected_supplementary_features = []
298     if remaining_features:
299         rfe_estimator = lgb.LGBMClassifier(random_state=42)
300         selector = RFE(estimator=rfe_estimator, n_features_to_select
301             =5, step=1)
302         X_for_rfe = pd.concat([meta_features, X_original], axis=1)
303         selector = selector.fit(X_for_rfe, y_multistate)
304
305         all_selected_names = X_for_rfe.columns[selector.support_]
306         selected_supplementary_features = [f for f in
307             all_selected_names if f in remaining_features]
308         print(f" > RFE额外选择出的补充特征: (
309             selected_supplementary_features)")
310
311     final_original_features = existing_core_features +
312         selected_supplementary_features
313
314     print("步骤 3: 构建最终的元模型训练集")
315     X_meta = pd.concat([meta_features, X_original[
316         final_original_features]], axis=1)
317
318     print("步骤 4: 使用Optuna优化元模型")
319     unique_classes = sorted(y_multistate.unique())
320     num_class = len(unique_classes)
321
322     def objective(trial):
323         params = {
324             'objective': 'multiclass', 'num_class': 8,
325             'metric': 'multi_logloss', 'verbosity': -1, 'n_estimators': 1000,
326             'learning_rate': trial.suggest_float('learning_rate',
327                 0.01, 0.2),
328             'num_leaves': trial.suggest_int('num_leaves', 20, 200),
329             'max_depth': trial.suggest_int('max_depth', 3, 10),
330         }
331         skf_meta = StratifiedKFold(n_splits=5, shuffle=True,
332             random_state=42)
333         logloss_scores = []

```

```

324     for train_index, val_index in skf_meta.split(X_meta,
325           y_multistate):
326         X_train_meta, y_train_meta = X_meta.iloc[train_index],
327           y_multistate.iloc[train_index]
328
329         class_counts = y_train_meta.value_counts()
330         minority_classes = class_counts[class_counts > 1]
331
332         if len(minority_classes) == len(class_counts):
333             X_train_smote, y_train_smote = X_train_meta,
334             y_train_meta
335         else:
336             sampling_strategy = {k: v for k, v in class_counts.
337               items() if k in minority_classes.index}
338             majority_count = class_counts.max()
339             for k in sampling_strategy.keys():
340                 if sampling_strategy[k] < majority_count:
341                     sampling_strategy[k] = max(sampling_strategy
342                       [k], int(majority_count * 0.5))
343             try:
344                 smote = SMOTE(random_state=42, k_neighbors=1,
345                   sampling_strategy=sampling_strategy)
346                 X_train_smote, y_train_smote = smote.fit_resample
347                   (X_train_meta, y_train_meta)
348             except Exception as e:
349                 print(f"SMOTE失败({e}), 本折回退到原始数据进行训
350                   练。")
351                 X_train_smote, y_train_smote = X_train_meta,
352                 y_train_meta
353
354             model = lgb.LGBMClassifier(**params)
355             model.fit(X_train_smote, y_train_smote,
356               eval_set=[(X_meta.iloc[val_index], y_multistate
357                 .iloc[val_index])],
358               eval_metric='multi_logloss',
359               callbacks=[lgb.early_stopping(50, verbose=False
360                 )])
361
362             preds = model.predict_proba(X_meta.iloc[val_index])
363             logloss_scores.append(log_loss(y_multistate.iloc[
364               val_index], preds, labels=unique_classes))
365
366     return np.mean(logloss_scores) if logloss_scores else 999.0
367
368 storage_path_meta = f"sqlite:///({OPTUNA_STUDIES_DIR} / '
369   optuna_meta_model.db' )"
370 study_meta = optuna.create_study(direction='minimize', study_name
371   ="meta_model_optimization", storage=storage_path_meta,
372   load_if_exists=True)
373 study_meta.optimize(objective, n_trials=n_trials,
374   show_progress_bar=True)
375 print(f"元模型找到的最佳LogLoss: {study_meta.best_value:.6f}")
376
377 print("步骤 5: 在全部数据上训练并保存最终元模型")
378 best_params_meta = study_meta.best_params
379 best_params_meta['num_class'] = num_class
380 final_meta_model = lgb.LGBMClassifier(**best_params_meta)

```



```

365     final_meta_model.fit(X_meta, y_multistate)
366
367     save_path_meta = MODELS_DIR / META_MODEL_NAME
368     with open(save_path_meta, 'wb') as f:
369         pickle.dump(final_meta_model, f)
370     print(f"元模型 '{META_MODEL_NAME}' 已保存至 '{save_path_meta}'")
371
372
373 # 4. 评估模块
374
375 def evaluate_final_system():
376     """对最终的诊断系统进行完整的5折交叉验证评估。"""
377
378     print("步骤 1: 加载建模数据")
379     df = pd.read_csv(MODELING_DATA_FILE)
380
381     print("步骤 2: 加载所有预训练模型")
382     try:
383         expert_models = {}
384         for key, config in EXPERT_MODELS_CONFIG.items():
385             model_path = MODELS_DIR / config['name']
386             if not model_path.exists():
387                 raise FileNotFoundError(f"找不到专家模型: {model_path}")
388             with open(model_path, 'rb') as f:
389                 expert_models[key] = pickle.load(f)
390
391         meta_model_path = MODELS_DIR / META_MODEL_NAME
392         if not meta_model_path.exists():
393             raise FileNotFoundError(f"找不到元模型: {meta_model_path}")
394         with open(meta_model_path, 'rb') as f:
395             meta_model = pickle.load(f)
396     except FileNotFoundError as e:
397         print(f"错误: {e}。请确保已成功运行 'train' 步骤。")
398     return
399
400     all_targets = TARGET_COLUMNS + [INTERACTION_TARGET]
401     X_original = df.drop(columns=[col for col in all_targets if col
402                                in df.columns], errors='ignore')
402     y_multistate = df[TARGET_COLUMNS[2]] * 4 + df[TARGET_COLUMNS[1]]
403     + 2 + df[TARGET_COLUMNS[0]] * 1
404
405     print("步骤 3: 执行5折交叉验证评估")
406     all_y_true, all_y_pred_final = [], []
407     skf_eval = StratifiedKFold(n_splits=5, shuffle=True, random_state
408                               =1337)
409
410     for fold, (train_index, val_index) in enumerate(skf_eval.split(
411         X_original, y_multistate)):
412         X_val = X_original.iloc[val_index]
413         y_val_true = y_multistate.iloc[val_index]
414
415         meta_features_val = pd.DataFrame(index=X_val.index)
416         for key, model in expert_models.items():
417             config = EXPERT_MODELS_CONFIG[key]
418             meta_features_val[f"{config['target']}_prob"] = model.

```

```

416         predict_proba(X_val)[:, 1]
417
418     # 从元模型对象中获取它训练时所用的特征列表
419     meta_model_features = meta_model.feature_name_
420
421     # 从X_val中只选取元模型需要的原始特征
422     original_features_for_meta = [f for f in meta_model_features
423                                   if f in X_val.columns]
424
425     # 构建元模型输入
426     X_meta_val = pd.concat([meta_features_val, X_val[
427                             original_features_for_meta]], axis=1)
428     # 确保列的顺序一致
429     X_meta_val = X_meta_val[meta_model_features]
430
431     y_pred_primary = meta_model.predict(X_meta_val)
432     y_pred_primary = pd.Series(y_pred_primary, index=X_val.index)
433
434     y_pred_logic = y_pred_primary.copy()
435
436     all_y_true.extend(y_val_true)
437     all_y_pred_final.extend(y_pred_logic)
438     print(f" - 第 {fold+1}/5 折评估完成。")
439
440     unique_classes_in_data = sorted(y_multistate.unique())
441     target_names = [STATE_LABELS.get(i, f"未知状态_{i}") for i in
442                     unique_classes_in_data]
443
444     print("\n最终分类结果:")
445     print(classification_report(all_y_true, all_y_pred_final,
446                                target_names=target_names, labels=unique_classes_in_data,
447                                zero_division=0))
448
449     conf_matrix = confusion_matrix(all_y_true, all_y_pred_final,
450                                    labels=unique_classes_in_data)
451     plt.figure(figsize=(12, 10))
452     sns.heatmap(conf_matrix, annot=True, fmt='g', cmap='viridis',
453                 xticklabels=target_names, yticklabels=target_names)
454     plt.title('最终诊断系统 - 汇总混淆矩阵')
455     plt.ylabel('真实状态')
456     plt.xlabel('预测状态')
457     plt.tight_layout()
458
459     FINAL_CONFUSION_MATRIX_PATH.parent.mkdir(parents=True, exist_ok=
460         True)
461     plt.savefig(FINAL_CONFUSION_MATRIX_PATH)
462     print(f"\n最终系统混淆矩阵图表已保存至 '{
463           FINAL_CONFUSION_MATRIX_PATH}'")
464
465 # 5. 流水线模块
466 def run_preprocessing():
467     """执行数据预处理步骤。"""
468     create_modeling_data()
469
470 def run_training():
471     """执行模型训练步骤 (专家模型 + 多分类元模型)。"""
472     train_all_expert_models()

```

```

464     train_meta_model()
465
466 def run_evaluation():
467     """执行最终模型评估步骤。"""
468     evaluate_final_system()
469
470 def run_full_pipeline():
471     """按顺序执行完整的数据预处理 -> 训练 -> 评估流程。"""
472     print("步骤 1: 数据预处理")
473     run_preprocessing()
474     print("步骤 2: 模型训练")
475     run_training()
476     print("步骤 3: 模型评估")
477     run_evaluation()
478     print("执行完毕")
479
480
481 # 6. 主函数
482
483 def main():
484     """主函数：解析命令行参数并执行相应步骤。"""
485     parser = argparse.ArgumentParser(description="CUMCM2025 Q4 智能诊
486         断系统（单文件版）")
487     parser.add_argument(
488         "step",
489         choices=["preprocess", "train", "evaluate", "all"],
490         help="选择要执行的流水线步骤：'preprocess' - 数据预处理，'
491             train' - 训练所有模型，'evaluate' - 评估模型，'all' - 运行
492             完整流程"
493     )
494
495     args = parser.parse_args()
496
497     if args.step == "preprocess":
498         print("开始步骤 1")
499         run_preprocessing()
500         print("完成步骤 1")
501
502     elif args.step == "train":
503         print("开始步骤 2")
504         run_training()
505         print("完成步骤 2")
506
507     elif args.step == "evaluate":
508         print("开始步骤 3")
509         run_evaluation()
510         print("完成步骤 3")
511
512     elif args.step == "all":
513         print("开始执行完整流水线")
514         run_full_pipeline()
515         print("完成完整流水线")
516
517 if __name__ == "__main__":
518     BASE_DIR.mkdir(parents=True, exist_ok=True)
519     main()

```
